Segmentação de Clientes de E-commerce: Análise RFM com Modelos Não Supervisionados

Nome: Leonardo Scholl Sternberg
Curso: Algoritmos para Agrupamento de Dados
Especialização em Inteligência Artificial para Engenheiros

1: Introdução

O presente projeto tem como objetivo realizar a segmentação de clientes de uma empresa de varejo online do Reino Unido, utilizando técnicas de aprendizado de máquina não supervisionado. A base de dados original consiste em registros transacionais de compras entre dezembro de 2010 e dezembro de 2011 ocorridas para uma empresa de varejo online sem loja física, sediada e registrada no Reino Unido. A empresa vende principalmente presentes exclusivos para todas as ocasiões, com muitos dos clientes da empresa são atacadistas. O dataset utilizado para o modelo é encontrado no seguinte link: https://www.kaggle.com/datasets/carrie1/ecommerce-data

O objetivo de projeto é agrupar os clientes com base em seus comportamentos de compra, a fim de identificar perfis valiosos como clientes "VIP", "Fiel", "Promissor" e clientes em "Risco", facilitando a tomada de decisão estratégica. Para isso, os dados brutos são transformados em métricas RFM (Recency, Frequency, Monetary) e submetidos a diferentes algoritmos de clusterização: K-means, DBSCAN, HDBSCAN e Método Hierárquico.

2: Metodologia

Nesta seção, são descritos os conceitos e métodos aplicados no tratamento dos dados e na modelagem, sem adentrar nos resultados obtidos.

2.1:Estrutura dos Dados

A abordagem metodológica tem início com o pré-processamento de uma base de dados transacional. A limpeza envolve a remoção de registros com valores inválidos (como quantidades e preços nulos ou negativos) e o tratamento de dados ausentes e duplicados. Após a limpeza, os dados são agregados por cliente para a criação da matriz RFM:

- **Recency (Recência):** Tempo (em dias) desde a última compra do cliente.
- **Frequency (Frequência):** Número total de compras únicas (faturas) realizadas.
- **Monetary (Monetário):** Valor total gasto pelo cliente.

Como as variáveis de comportamento financeiro geralmente apresentam alta assimetria e presença de *outliers*, aplica-se uma transformação logarítmica para suavizar as discrepâncias, seguida pela normalização por padronização (Z-score). Isso garante que todas as variáveis fiquem na mesma escala, requisito fundamental para algoritmos de agrupamento baseados em cálculo de distâncias.

Quanto aos agrupamentos/ clusters sugeridos, buscamos um valor de clusters maior que 2 pela representatividade para o negócio. Os cluster sugeridos são:

Cluster de clientes	**Descrição**	**Porcentagem estimada**
VIP	baixo recência, alto frequência, alto monetário	~10% da base
Fiel	baixo/ médio recência, médio/ alto frequência, médio/ alto monetário	~35% da base
Promissor	média recência, média frequência, média monetário	~25% da base
Risco	alta recência, baixa frequência, baixo monetário	~ 30% da base

2.2:Ambiente Python e Bibliotecas

O projeto é desenvolvido em ambiente Python (v3.14.3), fazendo o uso das seguintes bibliotecas principais para a fundamentação analítica:

- **Kagglehub (v1.0.0):** Importação do dataset de estudo.
- **Pandas (v3.0.1) e NumPy (v:2.4.2):** Para manipulação, limpeza e agregação de dados.
- **Matplotlib (v.3.10.8) e Seaborn (v.0.13.2):** Para a construção de gráficos e visualização de distribuições e relacionamentos entre variáveis.
- **Sweetviz (v.2.3.3):** Para a geração de relatórios automatizados de Análise Exploratória de Dados (EDA), facilitando o diagnóstico das variáveis antes e depois da limpeza.
- **Scikit-Learn (sklearn) (v1.8.0):** Principal motor para a normalização de dados e implementação dos modelos K-means e DBSCAN, além da extração de métricas de validação (Silhouette Score, Davies-Bouldin, Adjusted Rand Index).
- **SciPy (v.1.17.1):** Utilizada para a construção e visualização de dendrogramas aplicados aos métodos hierárquicos.

2.3:K-means

O K-means é um algoritmo de agrupamento baseado em centroides que busca particionar os dados em *K* clusters, minimizando a variância (inércia) dentro de cada grupo. O valor de *K* precisa ser definido a priori. Para encontrar o número ideal de clusters, utilizam-se técnicas de avaliação quantitativa:

- **Método do Cotovelo (Inércia):** Avalia a soma das distâncias quadradas intra-cluster, buscando o ponto onde a curva começa a suavizar.

- **Silhouette Score:** Mede a coesão (distância para os pontos do mesmo cluster) e a separação (distância para pontos de outros clusters). Valores mais altos indicam melhor agrupamento.
- **Davies-Bouldin Score:** Avalia a razão entre as distâncias intra-cluster e inter-cluster. Busca-se o menor valor possível.

2.4:DBSCAN

O DBSCAN (Density-Based Spatial Clustering of Applications with Noise) é um modelo de agrupamento espacial baseado em densidade. Diferente do K-means, ele não requer a definição prévia do número de clusters e é capaz de identificar dados atípicos como ruído (outliers). O método depende de dois hiperparâmetros fundamentais:

- **eps:** O raio de alcance (vizinhança) ao redor de um dado ponto. A escolha ideal de *eps* geralmente é guiada pelo Gráfico de K-Distâncias, observando o ponto de maior curvatura ("joelho").
- **min_samples:** O número mínimo de pontos que devem existir dentro da vizinhança definida por *eps* para que a região seja considerada densa (núcleo).

2.5:HDBSCAN

O HDBSCAN (Hierarchical DBSCAN) é uma extensão evolutiva do DBSCAN que o transforma em um processo hierárquico, extraindo clusters baseados em estabilidade de densidade. A grande vantagem metodológica do HDBSCAN é eliminar a necessidade de definir o parâmetro de distância fixa (*eps*). Seu principal hiperparâmetro de ajuste é o min_cluster_size (tamanho mínimo para que um grupo de pontos seja considerado um cluster isolado). A calibração deste modelo envolve a análise da proporção de ruído gerado e a maximização da métrica de *Silhouette* nos pontos não rotulados como ruído.

2.6:Método Ward

O Agrupamento Hierárquico Aglomerativo constrói uma hierarquia de clusters de baixo para cima, unindo iterativamente os pares de pontos ou grupos mais similares. A escolha de quais grupos fundir baseia-se em critérios de ligação (*linkage*). O método de Ward é a abordagem utilizada, pois visa minimizar a variância intra-cluster a cada passo de fusão. Outros critérios de ligação existem, como o Single (distância mínima entre grupos, que frequentemente sofre com encadeamento em formato de corrente) e o Complete (distância máxima entre grupos, forçando clusters altamente compactos). *Nota-se que os métodos Single e Complete não foram aplicados no escopo principal do projeto por não apresentarem resultados satisfatórios na separação dos dados.*

Para analisar a estrutura hierárquica, utiliza-se:

- **Dendrograma:** um diagrama em árvore que ilustra visualmente o processo de agrupamento hierárquico, exibindo como cada elemento ou cluster se funde aos demais e o nível de distância entre eles.

- **Método de Mojena:** um critério quantitativo aplicado ao dendrograma para determinar o número ideal de grupos, utilizando a média e o desvio padrão das distâncias de fusão para identificar saltos significativos na estrutura.
- **Regra de Parada de Mojena:** um método aplicado em cima do método de Mojena para determinar o melhor número de clusters baseado no fator de Mojena.

2.7:Comparação de Métodos

Para avaliar a concordância e a aderência das diferentes partições geradas (K-means, DBSCAN, HDBSCAN e Ward), a metodologia baseia-se em um conjunto de técnicas quantitativas e visuais de triangulação de modelos. Como os métodos possuem dinâmicas distintas (centroides, densidade e hierarquia), a comparação é fundamentada nas seguintes abordagens:

- **Heatmaps de Contingência (Matriz de Consistência):** Consiste na elaboração de tabelas de tabulação cruzada (*crosstab*) comparando as classificações de negócio de um modelo contra o outro. Essas matrizes são visualizadas através de mapas de calor, permitindo identificar visualmente se um cliente caracterizado como "VIP" por um algoritmo manteve esse comportamento no outro, facilitando o mapeamento de interseções.
- **Adjusted Rand Index (ARI):** É uma métrica estatística utilizada para medir a similaridade estrutural entre pares de segmentações diferentes. O índice avalia o quão concordantes são os agrupamentos, onde um ARI igual a 1,0 representa partições idênticas, valores próximos a zero indicam agrupamentos puramente aleatórios, e valores negativos apontam um desempenho pior que o acaso.
- **Comparativo de *Silhouette Score*:** Realiza-se o pareamento do índice final de Silhueta alcançado por cada modelo. Para garantir uma comparação metodologicamente coerente com a natureza dos algoritmos baseados em densidade (DBSCAN e HDBSCAN), o cálculo dessa métrica é ajustado para ignorar os pontos classificados como ruído (outliers), comparando apenas a coesão dos núcleos.
- **Distribuição Percentual e Volumétrica:** Utiliza-se uma tabela consolidada de contagens para comparar a quantidade absoluta de clientes e a proporção da base alocada em cada segmento de negócio (Fiel, Promissor, Risco, VIP e Outlier) por cada algoritmo. Essa visão valida a coerência da representatividade dos grupos formados frente às expectativas iniciais de negócio.

3: Resultados e discussão

Nesta seção, são descritos os resultados e discussão da implementação dos métodos previamente mencionados no dataset em questão, junto com visualização dos códigos.

3.1:Carregando o Dataset

Nesta etapa foi importado o dataset e as bibliotecas utilizadas no trabalho. O dataset bruto carregado continha 541.909 linhas e 8 colunas.

Carregando dataset

```python
# !/usr/bin/env python3
import kagglehub
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, HDBSCAN, AgglomerativeClustering
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score, davies_bouldin_score, adjusted_rand_score
from scipy.cluster.hierarchy import dendrogram, linkage

# Baixar dataset
path = kagglehub.dataset_download("carrie1/ecommerce-data")
df = pd.read_csv(f"{path}/data.csv", encoding='latin1')

print(f"Dataset carregado: {df.shape[0]} linhas e {df.shape[1]} colunas.")
```

✓ 3.6s

```
Dataset carregado: 541909 linhas e 8 colunas.
```

3.2:Análise dos Dados

Na análise exploratória diagnóstica, primeiro visualizamos a estrutura de dados, e encontramos a existência de 541909 entradas e 8 colunas. Também encontramos as seguintes colunas:

- InvoiceNo: Número da fatura (string)
- StockCode: Código do produto (string)
- Description: Descrição do produto (string)
- Quantity: Quantidade comprada (int)
- InvoiceDate: Data da fatura (string)
- UnitPrice: Preço unitário (float)
- CustomerID: ID do cliente (float)
- Country: País do cliente (string)

Análise dos Dados

Para uma compreensão mais profunda dos dados, visualizamos distribuições de variáveis-chave. Isso ajuda a identificar outliers, assimetrias e padrões iniciais antes do pré-processamento. Conseguimos ver:

- Identificação de dados duplicados
- Indeitificação de dados nulos
- Identificação de Ruídos Operacionais (UnitPrice < 0, Quantity < 0)
- Diagnóstico de Assimetria Extrema (Reino Unido dominando a base)

```python
# Ver as primeiras linhas para entender a "cara" dos dados
print(df.head())
```

✓ 0.0s

```
  InvoiceNo StockCode                          Description  Quantity  \
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
1    536365     71053                  WHITE METAL LANTERN         6
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

      InvoiceDate  UnitPrice  CustomerID         Country
0  12/1/2010 8:26       2.55     17850.0  United Kingdom
1  12/1/2010 8:26       3.39     17850.0  United Kingdom
2  12/1/2010 8:26       2.75     17850.0  United Kingdom
3  12/1/2010 8:26       3.39     17850.0  United Kingdom
4  12/1/2010 8:26       3.39     17850.0  United Kingdom
```

```python
# Ver tipos de dados e se há valores nulos óbvios
print(df.info())
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  str
 1   StockCode    541909 non-null  str
 2   Description  540455 non-null  str
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  str
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  str
dtypes: float64(2), int64(1), str(5)
memory usage: 33.1 MB
None
```

Verificamos a existência de 5.268 dados duplicados e a unicidade das colunas. Constatou-se também a presença de 135.080 valores ausentes na coluna essencial de identificação do cliente (CustomerID) e 1454 na coluna de descrição. A coluna descrição não é essencial para o algoritmo, mas o CustomerID necessitará tratamento.

```python
# Verificar se há linhas duplicadas
print(f"Duplicados: {df.duplicated().sum()}")
```

✓ 0.3s

```
Duplicados: 5268
```

```python
# Criando um resumo da cardinalidade
resumo_unique = pd.DataFrame({
    'Coluna': df.columns,
    'Tipo': df.dtypes.values,
    'Valores Únicos': df.nunique().values,
    '% de Unicidade': (df.nunique().values / len(df)) * 100
})

# Ordenar pelos valores únicos para facilitar a análise
print(resumo_unique.sort_values(by='Valores Únicos'))
```

✓ 0.3s

```
        Coluna     Tipo  Valores Únicos  % de Unicidade
7      Country      str              38        0.007012
3     Quantity    int64             722        0.133233
5    UnitPrice  float64            1630        0.300789
1    StockCode      str            4070        0.751049
2  Description      str            4223        0.779282
6   CustomerID  float64            4372        0.806778
4  InvoiceDate      str           23260        4.292234
0    InvoiceNo      str           25900        4.779400
```

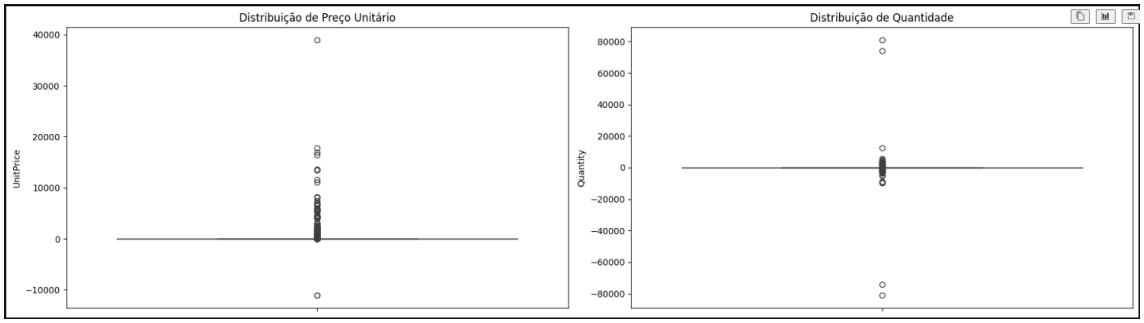
```python
# Verificar tipos de dados e valores nulos
print(df.info())
print("\nValores nulos por coluna:")
print(df.isnull().sum())
```

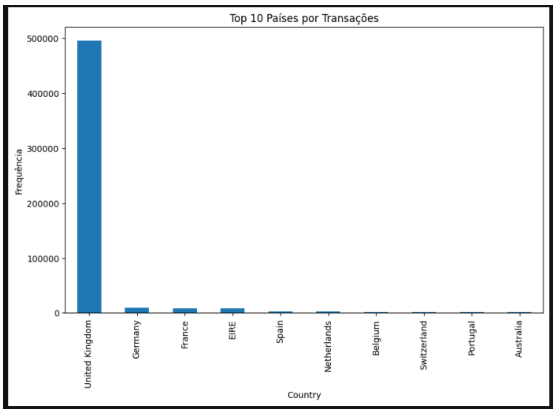
✓ 0.1s

```
<class 'pandas.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  str
 1   StockCode    541909 non-null  str
 2   Description  540455 non-null  str
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  str
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  str
dtypes: float64(2), int64(1), str(5)
memory usage: 33.1 MB
None

Valores nulos por coluna:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64
```

Foram identificados também ruídos operacionais críticos (valores negativos em quantidade e preços) e uma dominância extrema do Reino Unido no volume de vendas, caracterizando alta assimetria. A visualização por boxplots ressaltou a presença de severos *outliers* na distribuição de preços e quantidades.

```
# 1. Distribuição de Preço e Quantidade (identificando outliers e escalas)
plt.figure(figsize=(18, 5))

plt.subplot(1, 2, 1)
sns.boxplot(y=df['UnitPrice'])
plt.title('Distribuição de Preço Unitário')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['Quantity'])
plt.title('Distribuição de Quantidade')

plt.tight_layout()
plt.show()

# 2. Top 10 Países em volume de transações
plt.figure(figsize=(10, 6))
df['Country'].value_counts().head(10).plot(kind='bar')
plt.title('Top 10 Países por Transações')
plt.ylabel('Frequência')
plt.show()
```

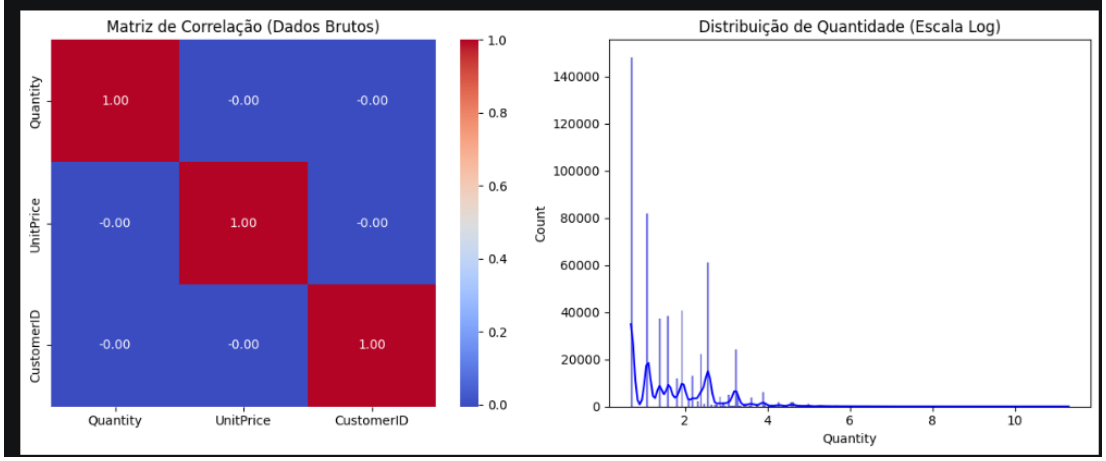




Foi feita a EDA Diagnóstica pré normalização e tratamento de dados, através de uma matriz de correlação entre variáveis numéricas (para garantir a não correlação entre elas) e visualizar como seria o campo quantidade na escala log. A distribuição na escala log é ótima para lidar com datasets com grande assimetria e quantidade de outliers.

```python
# EDA Diagnóstica (Antes do Pré-processamento)

# Selecionando apenas colunas numéricas para evitar erros de tipo
# O dropna aqui é apenas temporário para a visualização não quebrar
df[['Quantity', 'UnitPrice', 'CustomerID', 'InvoiceDate']].dropna()
df_temp = df[['Quantity', 'UnitPrice', 'CustomerID']]

plt.figure(figsize=(12, 5))

# 1. Matriz de Correlação: Verifica se as variáveis são independentes
plt.subplot(1, 2, 1)
sns.heatmap(df_temp.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriz de Correlação (Dados Brutos)')

# 2. Distribuição de Quantidade: Demonstra a assimetria e necessidade de escala
# Aplicamos log1p (logaritmo) apenas no gráfico para conseguir visualizar a densidade
plt.subplot(1, 2, 2)
sns.histplot(np.log1p(df_temp['Quantity']), kde=True, color='blue')
plt.title('Distribuição de Quantidade (Escala Log)')

plt.tight_layout()
plt.show()
```

```
c:\Users\leoss\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pandas\core\arraylike.py:402: RuntimeWarning: divide by zero encountered in log1p
  result = getattr(ufunc, method)(*inputs, **kwargs)
c:\Users\leoss\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pandas\core\arraylike.py:402: RuntimeWarning: invalid value encountered in log1p
  result = getattr(ufunc, method)(*inputs, **kwargs)
```



3.3:Pré-processamento Definitivo

A limpeza consistiu em remover valores nulos, faturas duplicadas e transações de estorno (onde Quantity e UnitPrice eram menores ou iguais a zero). Os dados foram agrupados para criar as variáveis de Recência, Frequência e Monetário (RFM). Para contornar a forte assimetria, aplicou-se uma transformação logarítmica seguida da normalização com o StandardScaler. O processo reduziu a base bruta para um conjunto final comportamental com 4338 clientes únicos.

Pré-processamento Definitivo (RFM)

- Eliminação de dados inválidos
- Tratamento de valores ausentes
- Conversão de tipos
- Criação de features
- Normalização
- Seleção de features

```python
# --- PRÉ-PROCESSAMENTO DEFINITIVO (RFM) ---

# 1. Eliminação de dados inválidos (Quantity e UnitPrice > 0, e valores ausentes) e duplicados
df_clean = df[(df['Quantity'] > 0) & (df['UnitPrice'] > 0)].copy()
df_clean.drop_duplicates(inplace=True)
df_clean.dropna(subset=['Quantity', 'UnitPrice', 'CustomerID' ,'InvoiceDate'], inplace=True)

# 2. Conversão de tipos
df_clean['InvoiceDate'] = pd.to_datetime(df_clean['InvoiceDate'])
df_clean['CustomerID'] = df_clean['CustomerID'].astype(int)

# 3. Criação de features (Valor Total e métricas RFM)
df_clean['TotalSum'] = df_clean['Quantity'] * df_clean['UnitPrice']
snapshot_date = df_clean['InvoiceDate'].max() + pd.DateOffset(days=1)

# Agregando por cliente para criar Recência, Frequência e Monetário
rfm = df_clean.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalSum': 'sum'
})

rfm.rename(columns={
    'InvoiceDate': 'Recency',
    'InvoiceNo': 'Frequency',
    'TotalSum': 'Monetary'
}, inplace=True)

# 4. Normalização (Log + StandardScaler para tratar assimetria)
# Aplicamos log para diminuir o peso dos outliers antes de normalizar
rfm_log = np.log1p(rfm)
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm_log)

# 5. Seleção de features para o Dataframe final
df_rfm_final = pd.DataFrame(rfm_scaled, index=rfm.index, columns=rfm.columns)

print(f"Base processada com sucesso. Total de clientes: {df_rfm_final.shape}")
print(df_rfm_final.head())
```

✓ 1.2s

```
Base processada com sucesso. Total de clientes: (4338, 3)
             Recency  Frequency  Monetary
CustomerID
12346       1.461993  -0.955214  3.707716
12347      -2.038734   1.074425  1.414903
12348       0.373104   0.386304  0.720024
12349      -0.623086  -0.955214  0.702287
12350       1.424558  -0.955214 -0.614514
```

3.4:EDA

Gerou-se relatórios exploratórios pré e pós-limpeza utilizando a biblioteca Sweetviz. Após a limpeza, o arquivo validou a consistência do *dataframe* processado, com 4338 linhas, nenhum registro duplicado e a garantia das 3 *features* numéricas estarem adequadamente normalizadas em escalas comparáveis, centradas próximo a zero. Os arquivos pré e pós limpeza estão em anexo.

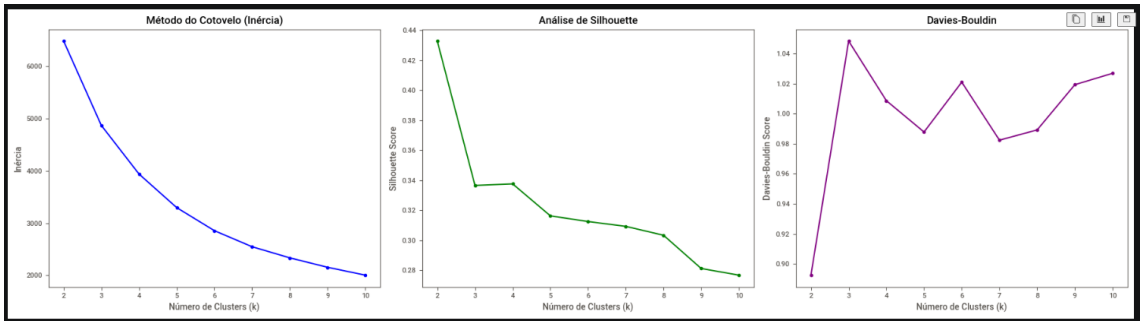
3.5:K-means

A implementação do algoritmo K-means iniciou-se com a determinação do número ideal de agrupamentos (*K*). Para isso, o algoritmo foi testado iterativamente em um intervalo de *K*=2 a *K*=10, avaliando-se três métricas quantitativas. A análise conjunta dos resultados demonstrou que não havia um "cotovelo" óbvio na curva de Inércia. O índice de *Silhouette* apresentou os melhores patamares em 3 e 4, mantendo-se viável em 5, enquanto o índice de Davies-Bouldin atingiu mínimos locais ideais em 5 e 7. Buscando o melhor equilíbrio entre o rigor matemático e a granularidade necessária para a análise, optou-se pela utilização de K=5.

K-means

- Método do cotovelo: pegamos valores quando a curva tende a suavizar/ inércia
- Silhouette: tentamos pegar o valor máximo
- Davies–Bouldin: tentamos pegar melhor valor

```python
# --- MODELAGEM E AVALIAÇÃO QUANTITATIVA ---

# 1. Calculando Inércia, Silhouette e Davies-Bouldin para diferentes valores de K
inertia = []
silhouette_vals = []
davies_bouldin_vals = []
k_range = range(2, 11) # Testando de 2 a 10 clusters

for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(df_rfm_final)
    inertia.append(kmeans.inertia_)
    silhouette_vals.append(silhouette_score(df_rfm_final, kmeans.labels_))
    davies_bouldin_vals.append(davies_bouldin_score(df_rfm_final, kmeans.labels_))

# 2. Visualização dos resultados para decisão de K
plt.figure(figsize=(18, 5))

# Gráfico do Cotovelo (Inércia)
plt.subplot(1, 3, 1)
plt.plot(k_range, inertia, marker='o', color='blue')
plt.title('Método do Cotovelo (Inércia)')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Inércia')

# Gráfico do Silhouette Score
plt.subplot(1, 3, 2)
plt.plot(k_range, silhouette_vals, marker='o', color='green')
plt.title('Análise de Silhouette')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Silhouette Score')

# Davies-Bouldin (menor é melhor)
plt.subplot(1, 3, 3)
plt.plot(k_range, davies_bouldin_vals, marker='o', color='purple')
plt.title('Davies-Bouldin')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Davies-Bouldin Score')

plt.tight_layout()
plt.show()
```



Com o modelo configurado para 5 agrupamentos, os dados da matriz RFM foram particionados e seus centroides (médias) foram extraídos para análise.

```python
# --- EXECUÇÃO K-MEANS (K=5) ---
k5 = 5
kmeans5 = KMeans(n_clusters=k5, init='k-means++', random_state=42, n_init=10)

# Criando colunas específicas
df_rfm_final['Cluster'] = kmeans5.fit_predict(df_rfm_final[['Recency', 'Frequency', 'Monetary']])
rfm['Cluster'] = df_rfm_final['Cluster']

# Avaliação Qualitativa
perfil = df_rfm_final.groupby('Cluster').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'Cluster': 'count'
}).rename(columns={'Cluster': 'Total_Clientes'})

print(perfil)
```

✓ 0.3s

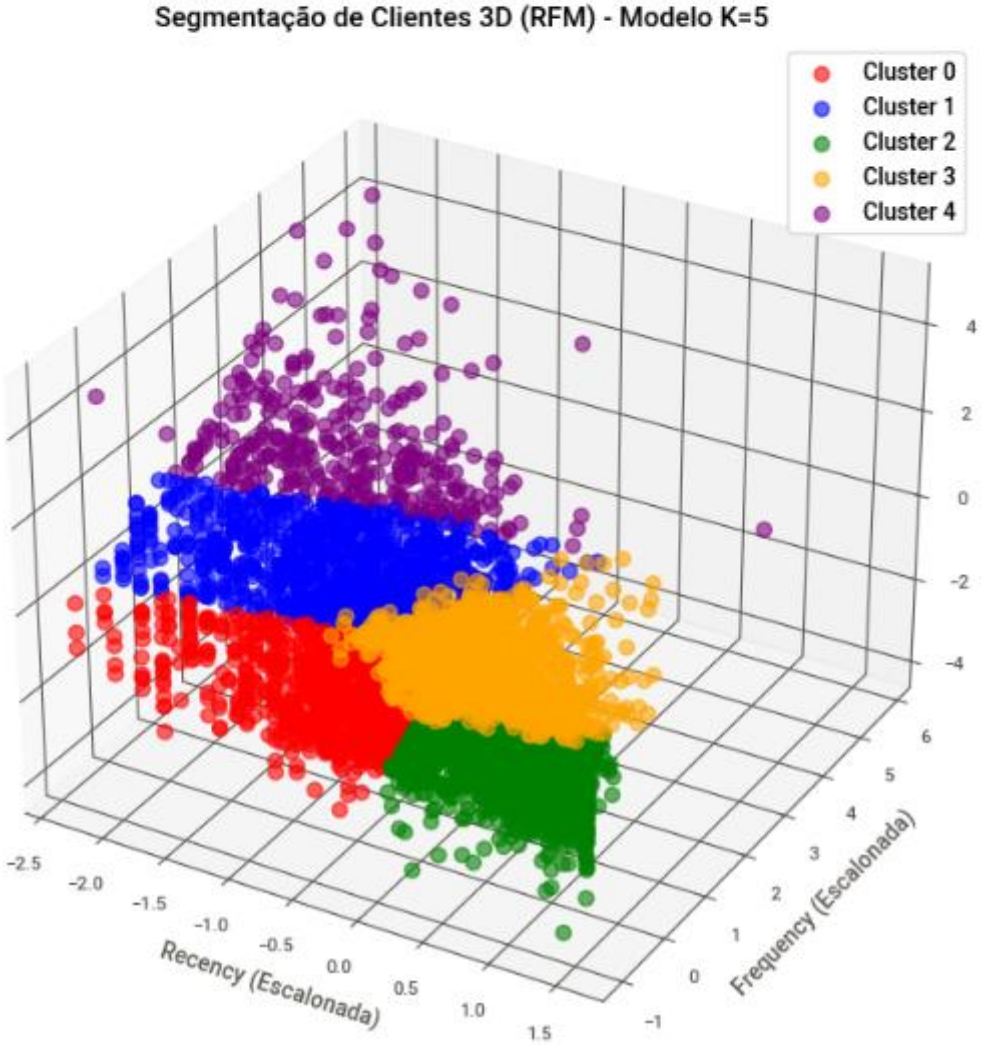
```
          Recency  Frequency  Monetary  Total_Clientes
Cluster
0       -0.540337  -0.599913 -0.634479             834
1       -0.890659   0.766277  0.731211             938
2        1.033577  -0.835743 -0.923325            1209
3        0.476359   0.018411  0.320861            1018
4       -1.322860   2.280918  1.867087             339
```

A visualização espacial tridimensional (Gráfico 3D) e os gráficos de densidade mútua (*Pairplot*) permitiram interpretar as características de recência, frequência e valor monetário de cada grupo. A partir dessa avaliação matemática, procedeu-se com a qualificação qualitativa, fundindo os 5 clusters matemáticos nos 4 perfis estratégicos de negócio predefinidos:

- O Cluster 4 destacou-se pela menor recência (compras muito recentes) combinada com frequência e valor monetário muito acima da média, caracterizando claramente o segmento VIP.
- Os Clusters 1 e 3 apresentaram bons volumes de frequência e valor, com recência variando entre média e baixa, sendo ambos unificados para compor o segmento de clientes Fiéis.
- O Cluster 0 obteve métricas intermediárias e ligeiramente abaixo da média, caracterizando o cliente Promissor, que possui engajamento mediano.
- O Cluster 2 isolou os clientes com as maiores taxas de recência (muito tempo inativos) e os menores engajamentos financeiros, configurando o alerta de Risco.

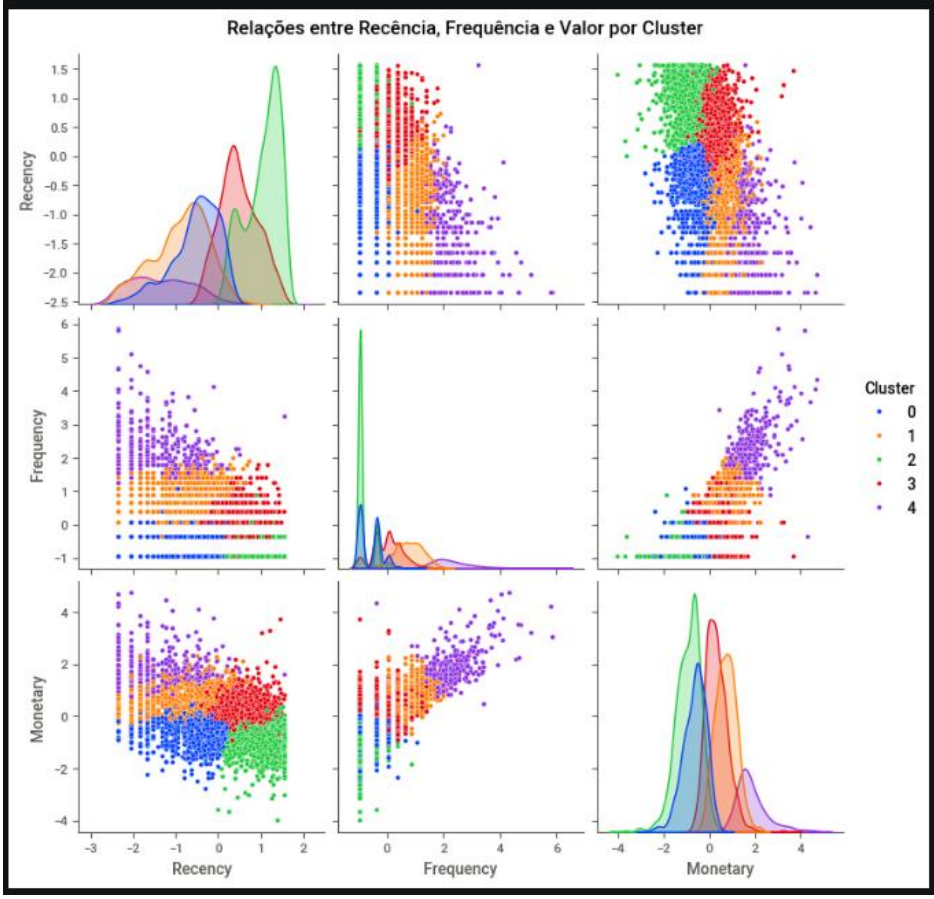
```python
# --- VISUALIZAÇÃO 3D ---

# Configuração da figura 3D
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Definindo as cores para os 5 clusters
cores_k5 = ['red', 'blue', 'green', 'orange', 'purple']

for i in range(k5):
    ax.scatter(df_rfm_final[df_rfm_final['Cluster'] == i]['Recency'],
               df_rfm_final[df_rfm_final['Cluster'] == i]['Frequency'],
               df_rfm_final[df_rfm_final['Cluster'] == i]['Monetary'],
               c=cores_k5[i], label=f'Cluster {i}', s=40, alpha=0.6)

ax.set_xlabel('Recency (Escalonada)')
ax.set_ylabel('Frequency (Escalonada)')
ax.set_zlabel('Monetary (Escalonada)')
ax.set_title('Segmentação de Clientes 3D (RFM) - Modelo K=5')
ax.legend()
plt.show()
```

✓ 0.2s



```python
# Gerando o gráfico de pares
sns.pairplot(df_rfm_final[['Recency', 'Frequency', 'Monetary', 'Cluster']], hue='Cluster', palette='bright', diag_kind='kde')
plt.suptitle('Relações entre Recência, Frequência e Valor por Cluster', y=1.02)
plt.show()
```

✓ 1.2s

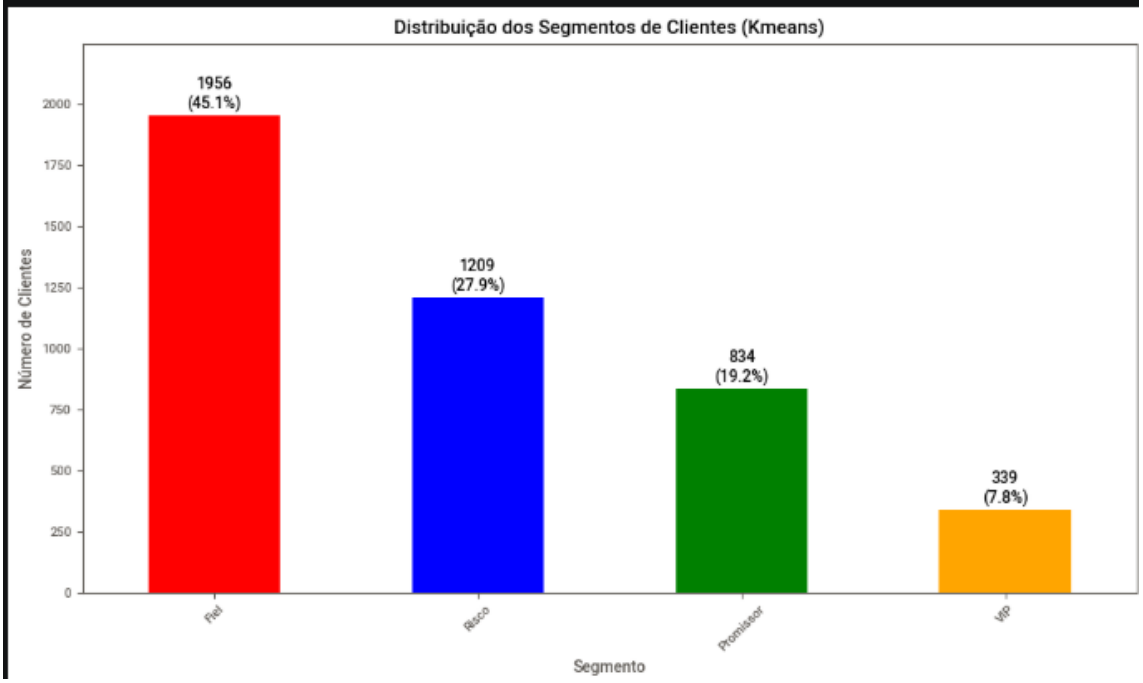



```python
# Mapeamento qualitativo
nomes_clusters_kmeans = {
    4: 'VIP',
    1: 'Fiel',
    3: 'Fiel',
    2: 'Risco',
    0: 'Promissor'
}

df_rfm_final['Segmento_KMeans'] = df_rfm_final['Cluster'].map(nomes_clusters_kmeans)
```

A distribuição volumétrica final da base de clientes através do modelo de K-means consolidou a seguinte proporção: Fiel com 45,1% (1.956 clientes), seguido por Risco com 27,9% (1.209 clientes), Promissor com 19,2% (834 clientes) e a base VIP representando 7,8% (339 clientes).

```python
# Ver em forma de gráfica
plt.figure(figsize=(10, 6))

# Calculamos as duas métricas
contagem = df_rfm_final['Segmento_KMeans'].value_counts()
porcentagem = df_rfm_final['Segmento_KMeans'].value_counts(normalize=True) * 100

# Criamos o gráfico principal (usando os números absolutos no eixo Y)
contagem.plot(kind='bar', color=cores_k5)

# Adicionamos os rótulos combinados (Número + %) no topo de cada barra
for i, (label, valor_absoluto) in enumerate(contagem.items()):
    pct = porcentagem[label]
    texto = f"{valor_absoluto}\n({pct:.1f}%)" # \n quebra a linha para ficar um sobre o outro
    plt.text(i, valor_absoluto + 10, texto, ha='center', va='bottom', fontweight='bold', fontsize=10)

plt.title('Distribuição dos Segmentos de Clientes (Kmeans)')
plt.xlabel('Segmento')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)

# Aumentar um pouco o limite do eixo Y para o texto não cortar no topo
plt.ylim(0, contagem.max() * 1.15)

plt.tight_layout()
plt.show()
```

✓ 0.0s



3.6:DBSCAN

A implementação do DBSCAN iniciou-se com a definição dos hiperparâmetros fundamentais: min_samples e eps. Por padrão, o min_samples foi estipulado em 6 (duas vezes o número de *features*). Para determinar o raio da vizinhança ideal (eps), realizou-se uma análise prévia do percentual de ruído gerado por diferentes valores, combinada com a plotagem do Gráfico de K-Distâncias.

```python
# Medindo ruído
def testar_dbscan(df, eps_values, min_samples=6):
    results = []
    features = df[['Recency', 'Frequency', 'Monetary']]

    for eps in eps_values:
        db = DBSCAN(eps=eps, min_samples=min_samples).fit(features)
        labels = db.labels_

        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        n_noise = list(labels).count(-1)
        perc_noise = (n_noise / len(df)) * 100

        results.append({
            'eps': eps,
            'min_samples': min_samples,
            'n_clusters': n_clusters,
            'n_noise': n_noise,
            'perc_noise': round(perc_noise, 2)
        })

    return pd.DataFrame(results)

# Definindo intervalos de teste (baseado na escala Z-score dos seus dados)
eps_range = [0.2, 0.3, 0.35, 0.4, 0.45, 0.5, 0.6, 0.7]

df_testes = testar_dbscan(df_rfm_final, eps_range)

# Mostrando apenas combinações que geraram mais de 2 clusters
print(df_testes.sort_values(by='n_noise', ascending=True))
```

✓ 0.2s

```
    eps  min_samples  n_clusters  n_noise  perc_noise
7  0.70            6           1       32        0.74
6  0.60            6           1       41        0.95
5  0.50            6           3       64        1.48
4  0.45            6           2       81        1.87
3  0.40            6           3      121        2.79
2  0.35            6           3      175        4.03
1  0.30            6          10      297        6.85
0  0.20            6          31      770       17.75
```

```python
# Gráfico de K-Distâncias para eps=0.35
n_neighbors = 6

neighbors = NearestNeighbors(n_neighbors=n_neighbors)
neighbors_fit = neighbors.fit(df_rfm_final[['Recency', 'Frequency', 'Monetary']])
distances, indices = neighbors_fit.kneighbors(df_rfm_final[['Recency', 'Frequency', 'Monetary']])

# Ordenando as distâncias
distances = np.sort(distances[:, 4], axis=0)

# Plotando
plt.figure(figsize=(8, 5))
plt.plot(distances)
plt.title(f'Gráfico de K-Distâncias para encontrar o eps com n_neighbors={n_neighbors}')
plt.xlabel('Pontos ordenados')
plt.ylabel('Distância')
plt.grid(True)
plt.show()
```

✓ 0.0s

Com base na visualização da curvatura ("joelho") do gráfico de distâncias e buscando equilibrar um bom número de clusters sem gerar ruído excessivo, o parâmetro ideal foi definido em eps=0.35.

```python
# Definido eps=0.35 pois possui bons números de clusters, minimiza ruído, e apresenta bom equilíbrio no gráfico de K-Distâncias
# Execução final do DBSCAN para comparação
dbscan_final = DBSCAN(eps=0.35, min_samples=6)
df_rfm_final['Cluster_DBSCAN'] = dbscan_final.fit_predict(df_rfm_final[['Recency', 'Frequency', 'Monetary']])

# Verificando as médias do DBSCAN para criar os nomes
perfil_dbscan = df_rfm_final.groupby('Cluster_DBSCAN').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'Cluster_DBSCAN': 'count' # Apenas para ver quantos clientes há em cada um
}).rename(columns={'Cluster_DBSCAN': 'Total_Clientes'})

print(perfil_dbscan)
```

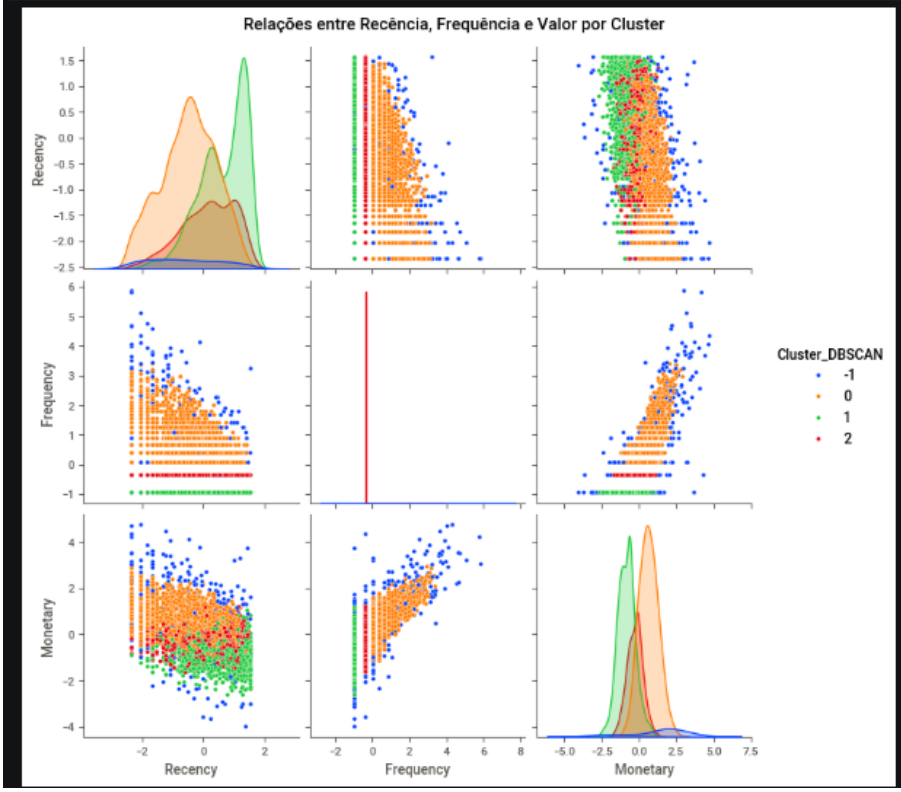
✓ 0.0s

```
                 Recency  Frequency  Monetary  Total_Clientes
Cluster_DBSCAN
-1             -0.604823   1.336217  1.194165             175
 0             -0.491339   0.778198  0.648977            1878
 1              0.626018  -0.955214 -0.826579            1464
 2              0.136527  -0.361583 -0.265099             821
```

A aplicação do algoritmo separou os dados em 3 agrupamentos de densidade (0, 1 e 2) e 1 grupo de ruído (-1). A partir do *pairplot* das *features* RFM e da análise dos valores médios de cada grupo gerado, realizou-se a qualificação dos perfis com base na regra de negócio:

- O cluster 0 apresentou recência média/baixa com alta frequência e valor monetário, caracterizando o Fiel.
- O cluster 1 apresentou alta recência com baixa frequência e valor monetário, alinhando-se ao perfil de Risco.
- O cluster 2 demonstrou baixa recência e métricas médias/baixas nas demais, indicando perfil Promissor.

```python
# Gerando o gráfico de pares
sns.pairplot(df_rfm_final[['Recency', 'Frequency', 'Monetary', 'Cluster_DBSCAN']], hue='Cluster_DBSCAN', palette='bright', diag_kind='kde')
plt.suptitle('Relações entre Recência, Frequência e Valor por Cluster', y=1.02)
plt.show()
```
✓ 1.3s



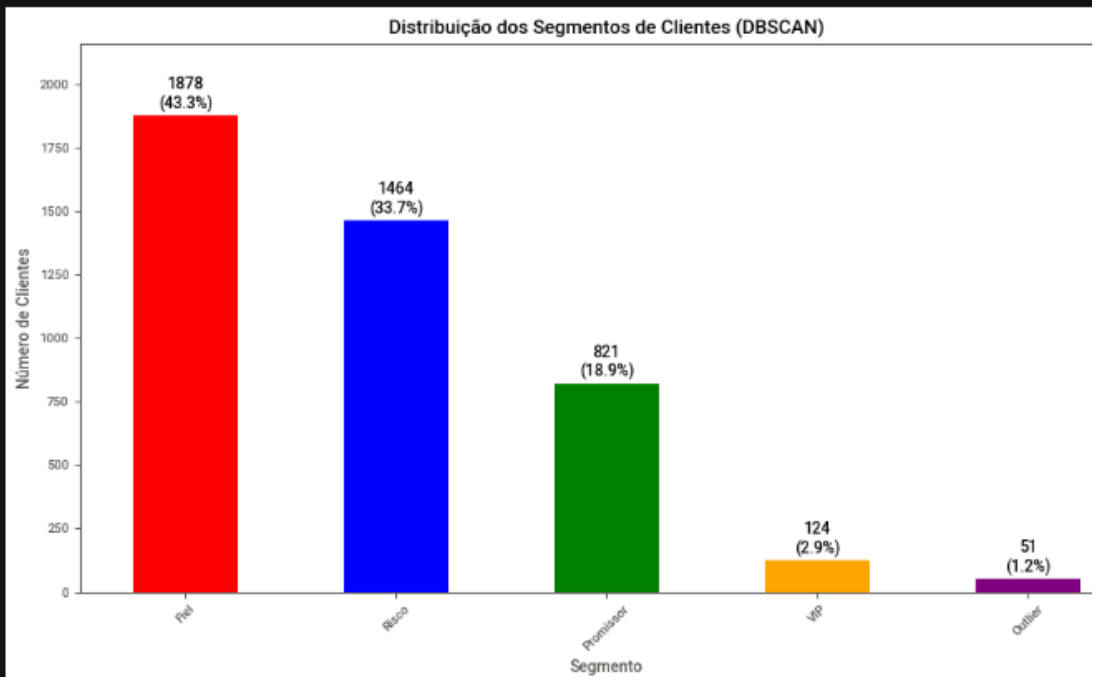
Os pontos não agrupados (cluster -1) passaram por um filtro adicional de negócio. Clientes atípicos, mas que possuíam valor Monetário superior ao percentil 75% da base, foram resgatados e reclassificados como **"VIP"**, enquanto o restante permaneceu classificado estritamente como **"Outlier"** (ruído).

```python
# Mapeamento qualitativo
nomes_clusters_dbscan = {
    -1: 'Outlier',
    0: 'Fiel',
    2: 'Promissor',
    1: 'Risco'
}

# Aplicando o mapeamento
df_rfm_final['Segmento_DBSCAN'] = df_rfm_final['Cluster_DBSCAN'].map(nomes_clusters_dbscan)

# Buscando os clinetes VIPs (outliers bons)
condicao_vip = (df_rfm_final['Cluster_DBSCAN'] == -1) & (df_rfm_final['Monetary'] > df_rfm_final['Monetary'].quantile(0.75))
df_rfm_final.loc[condicao_vip, 'Segmento_DBSCAN'] = 'VIP'
```

A visualização gráfica da distribuição final demonstrou a seguinte segmentação: Fiel (43,3%), Risco (33,7%), Promissor (18,9%), VIP (2,9%) e Outlier (1,2%).

```python
# Ver em forma de gráfica
plt.figure(figsize=(10, 6))

# Calculamos as duas métricas
contagem = df_rfm_final['Segmento_DBSCAN'].value_counts()
porcentagem = df_rfm_final['Segmento_DBSCAN'].value_counts(normalize=True) * 100

# Criamos o gráfico principal (usando os números absolutos no eixo Y)
contagem.plot(kind='bar', color=cores_k5)

# Adicionamos os rótulos combinados (Número + %) no topo de cada barra
for i, (label, valor_absoluto) in enumerate(contagem.items()):
    pct = porcentagem[label]
    texto = f"{valor_absoluto}\n({pct:.1f}%)" # \n quebra a linha para ficar um sobre o outro
    plt.text(i, valor_absoluto + 10, texto, ha='center', va='bottom', fontweight='bold', fontsize=10)

plt.title('Distribuição dos Segmentos de Clientes (DBSCAN)')
plt.xlabel('Segmento')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)

# Aumentar um pouco o limite do eixo Y para o texto não cortar no topo
plt.ylim(0, contagem.max() * 1.15)

plt.tight_layout()
plt.show()
```

✓ 0.0s



3.7:HDBSCAN

A modelagem com o HDBSCAN teve como foco principal a otimização de seu hiperparâmetro mais sensível: o min_cluster_size (número mínimo de dados para formação de um cluster válido). Para encontrar o valor ideal, o algoritmo foi submetido a um conjunto de testes iterativos com valores variando progressivamente. A avaliação dessa calibragem baseou-se em: número de clusters, a estabilização do *Silhouette Score* (calculado exclusivamente sobre os pontos que não eram ruído) e a proporção percentual de clientes classificados como ruído/outliers (cluster -1).

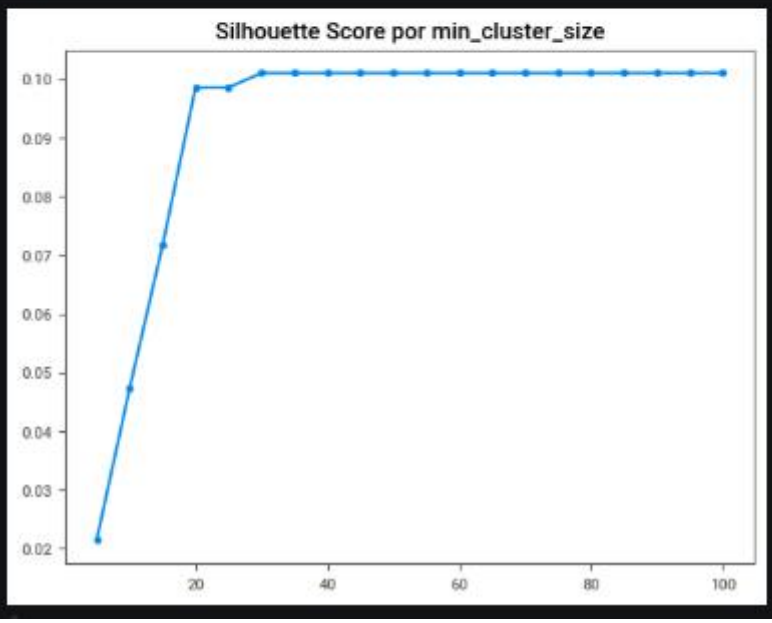
```python
# Analise de ruido
def testar_hdbscan(df, min_cluster_size_values):
    results = []
    scores = []
    features = df[['Recency', 'Frequency', 'Monetary']]

    for mcs in min_cluster_size_values:
        # Inicializando o HDBSCAN
        hdb = HDBSCAN(min_cluster_size=mcs, copy=True).fit(features)
        labels = hdb.labels_

        # Número de clusters (ignora o ruído -1)
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        # Contagem de ruído
        n_noise = list(labels).count(-1)
        # Percentual de ruído (importante para decidir se o modelo está sendo rígido demais)
        perc_noise = (n_noise / len(df)) * 100

        # Silhouette só funciona se houver mais de 1 cluster e ignora o ruído (-1)
        mask = labels != -1
        if len(set(labels[mask])) > 1:
            score = silhouette_score(features[mask], labels[mask])
            scores.append(score)
        else:
            scores.append(0)

        results.append({
            'min_cluster_size': mcs,
            'n_clusters': n_clusters,
            'n_noise': n_noise,
            'perc_noise': round(perc_noise, 2),
            'silhouette_score': round(scores[-1], 3) if scores else 0
        })

    return pd.DataFrame(results)

min_cluster_size = range(10, 101, 5)

df_testes_hdb = testar_hdbscan(df_rfm_final, min_cluster_size)

print(df_testes_hdb.sort_values(by='perc_noise', ascending=True))
```

✓ 9.0s

	min_cluster_size	n_clusters	n_noise	perc_noise	silhouette_score
6	40	2	467	10.77	0.319
7	45	2	534	12.31	0.323
2	20	3	584	13.46	0.191
8	50	2	594	13.69	0.327
9	55	2	656	15.12	0.330
10	60	2	702	16.18	0.333
3	25	3	740	17.06	0.199
11	65	2	753	17.36	0.335
1	15	4	798	18.40	0.145
12	70	2	818	18.86	0.338
0	10	5	828	19.09	0.124
13	75	2	878	20.24	0.341
4	30	3	882	20.33	0.205
14	80	2	952	21.95	0.345
5	35	3	1009	23.26	0.208
15	85	2	1013	23.35	0.349
16	90	2	1074	24.76	0.353
17	95	2	1129	26.03	0.356
18	100	2	1186	27.34	0.360

```python
features = df_rfm_final[['Recency', 'Frequency', 'Monetary']]
min_cluster_size = range(5, 101, 5)
scores = []

for mcs in min_cluster_size:
    model = HDBSCAN(min_cluster_size=mcs, min_samples=6, copy=True)
    labels = model.fit_predict(features)

    # Silhouette só funciona se houver mais de 1 cluster e ignora o ruído (-1)
    mask = labels != -1
    if len(set(labels[mask])) > 1:
        score = silhouette_score(features[mask], labels[mask])
        scores.append(score)
    else:
        scores.append(0)

# O pico deste gráfico indica o melhor mcs
plt.plot(min_cluster_size, scores, marker='o')
plt.title('Silhouette Score por min_cluster_size')
plt.show()
```

✓ 4.7s



A análise conjunta dos resultados demonstrou que o valor de min_cluster_size=20 representava o ponto de equilíbrio ótimo. Nesta configuração, o *Silhouette Score* atingiu um platô de máxima estabilidade e a quantidade de ruído gerada manteve-se controlada (aproximadamente 13,46% da base inicial). A aplicação do modelo com este hiperparâmetro resultou na divisão dos dados em 3 agrupamentos de densidade estáveis (0, 1 e 2) e 1 grupo de ruído (-1)

```python
# Pegamos min_cluster_size=20, pois atingiu um bom número de clusters e um percentual de ruído baixo, além de ser um ponto bom para o gráfico de Silhouette

# 1. Redefinindo as variáveis necessárias (garante que não dê erro de 'not defined')
rfm_cols = ['Recency', 'Frequency', 'Monetary']
X_all = df_rfm_final[rfm_cols]

# 2. Configurando o modelo baseado no seu teste vencedor (Linha 3 da sua tabela)
# min_cluster_size=250 e min_samples=2 geraram 5 clusters estáveis
modelo_hdb = HDBSCAN(
    min_cluster_size=20,
    copy=True
)

# 3. Aplicando o modelo
df_rfm_final['Cluster_HDBSCAN'] = modelo_hdb.fit_predict(X_all)

# 4. Criando o Perfil para análise

perfil_hdb = df_rfm_final.groupby('Cluster_HDBSCAN').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'Cluster_HDBSCAN': 'count' # Apenas para ver quantos clientes há em cada um
}).rename(columns={'Cluster_HDBSCAN': 'Total_Clientes'})

print(perfil_hdb)
```

✓ 0.0s

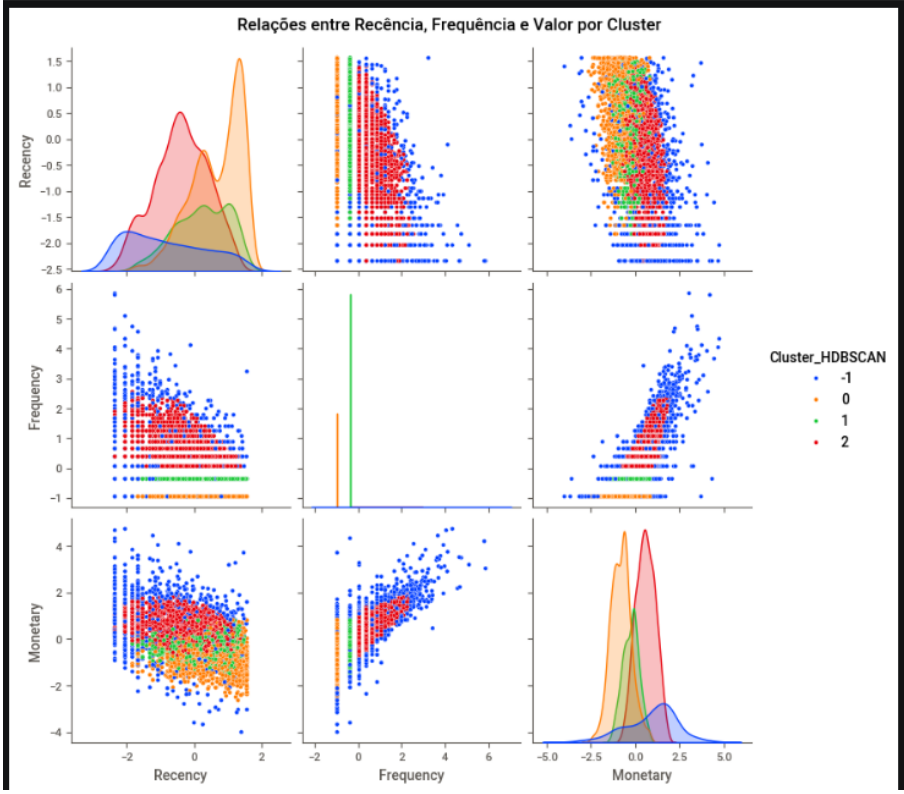
```
                  Recency  Frequency  Monetary  Total_Clientes
Cluster_HDBSCAN
-1               -0.831174   1.035359  0.883637             584
 0                0.646333  -0.955214 -0.827804            1443
 1                0.216992  -0.361583 -0.266492             748
 2               -0.389996   0.668067  0.561620            1563
```

Através da visualização da densidade no *pairplot* das *features* RFM e da extração das médias de cada agrupamento, procedeu-se com a rotulagem de negócio da mesma maneira que nos algoritmos anteriores:

- O cluster 2 destacou-se por uma recência média/baixa, combinada com alta frequência e métricas financeiras de médias a altas, caracterizando a base de clientes Fiéis.
- O cluster 1 apresentou recência baixa, com frequência e monetário de níveis médios a baixos, alinhando-se ao perfil do cliente Promissor.
- O cluster 0 isolou clientes com alta recência (muito tempo inativos) e níveis muito baixos de frequência e gasto, configurando claramente o segmento de Risco.

```python
# Gerando o gráfico de pares
sns.pairplot(df_rfm_final[['Recency', 'Frequency', 'Monetary', 'Cluster_HDBSCAN']], hue='Cluster_HDBSCAN', palette='bright', diag_kind='kde')
plt.suptitle('Relações entre Recência, Frequência e Valor por Cluster', y=1.02)
plt.show()
```

✓ 1.5s



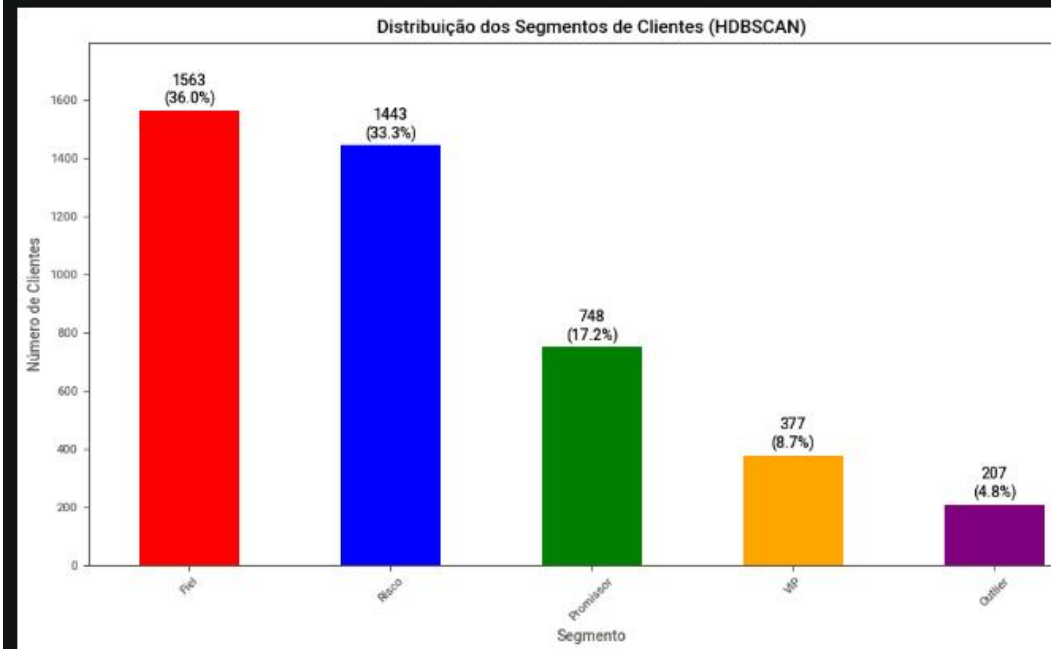
Assim como estabelecido na metodologia do modelo espacial anterior, os clientes não agrupados (cluster de ruído -1) passaram por uma verificação de negócio. Aqueles com comportamento atípico, mas que possuíam valor Monetário superior ao percentil 75% da base, foram isolados e reclassificados como perfil "VIP", restando no segmento "Outlier" apenas as anomalias estritas de baixo valor.

```python
# Mapeamento qualitativo
nomes_clusters_hdbscan = {
    -1: 'Outlier',
    2: 'Fiel',
    1: 'Promissor',
    0: 'Risco'
}

# Aplicando o mapeamento
df_rfm_final['Segmento_HDBSCAN'] = df_rfm_final['Cluster_HDBSCAN'].map(nomes_clusters_hdbscan)

# Buscando os clinetes VIPs (outliers bons)
condicao_vip = (df_rfm_final['Cluster_HDBSCAN'] == -1) & (df_rfm_final['Monetary'] > df_rfm_final['Monetary'].quantile(0.75))
df_rfm_final.loc[condicao_vip, 'Segmento_HDBSCAN'] = 'VIP'
```

A consolidação volumétrica do HDBSCAN apresentou a seguinte segmentação estratégica da base: Fiel com 36,0% (1.563 clientes) e Risco com 33,3% (1.443 clientes) englobando a maior fatia, seguidos por Promissor com 17,2% (748 clientes), VIP com 8,7% (377 clientes) e, por fim, as anomalias rotuladas como Outliers com 4,8% (207 clientes).

```python
# Ver em forma de gráfica
plt.figure(figsize=(10, 6))

# Calculamos as duas métricas
contagem = df_rfm_final['Segmento_HDBSCAN'].value_counts()
porcentagem = df_rfm_final['Segmento_HDBSCAN'].value_counts(normalize=True) * 100

# Criamos o gráfico principal (usando os números absolutos no eixo Y)
contagem.plot(kind='bar', color=cores_k5)

# Adiciona7557
# mos os rótulos combinados (Número + %) no topo de cada barra
for i, (label, valor_absoluto) in enumerate(contagem.items()):
    pct = porcentagem[label]
    texto = f"{valor_absoluto}\n({pct:.1f}%)" # \n quebra a linha para ficar um sobre o outro
    plt.text(i, valor_absoluto + 10, texto, ha='center', va='bottom', fontweight='bold', fontsize=10)

plt.title('Distribuição dos Segmentos de Clientes (HDBSCAN)')
plt.xlabel('Segmento')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)

# Aumentar um pouco o limite do eixo Y para o texto não cortar no topo
plt.ylim(0, contagem.max() * 1.15)

plt.tight_layout()
plt.show()
```

✓ 0.1s

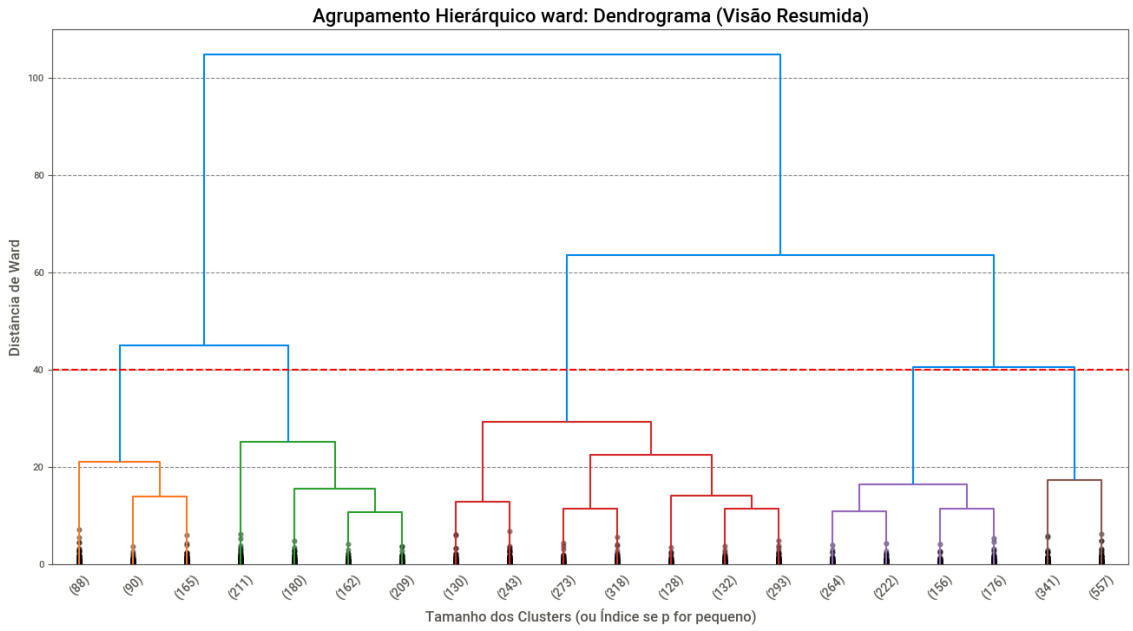


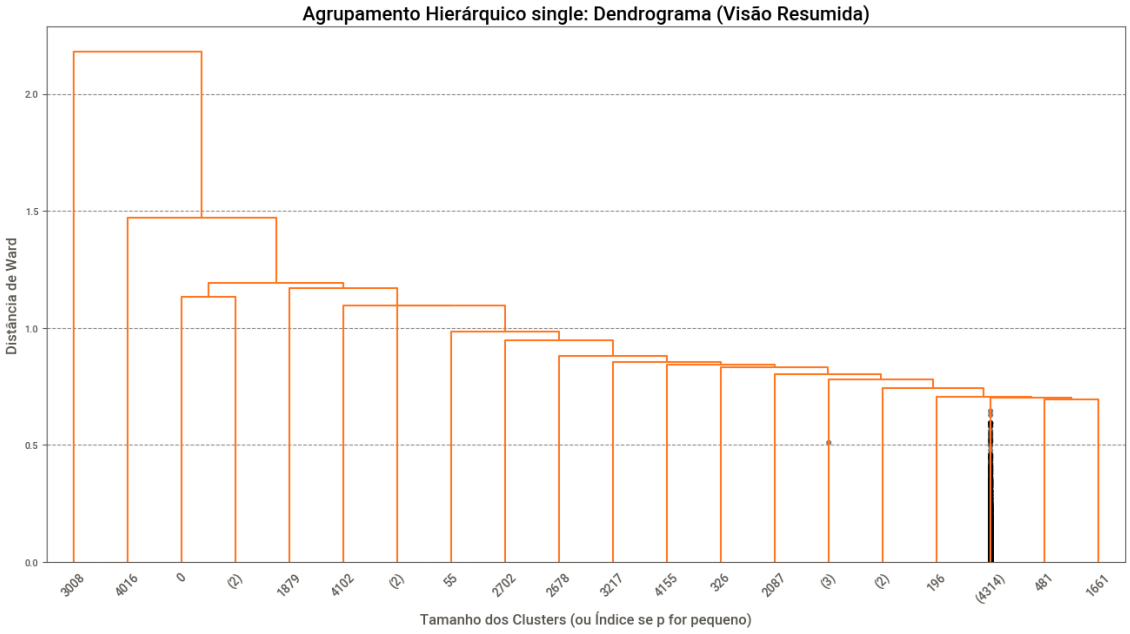
3.8:Método Ward

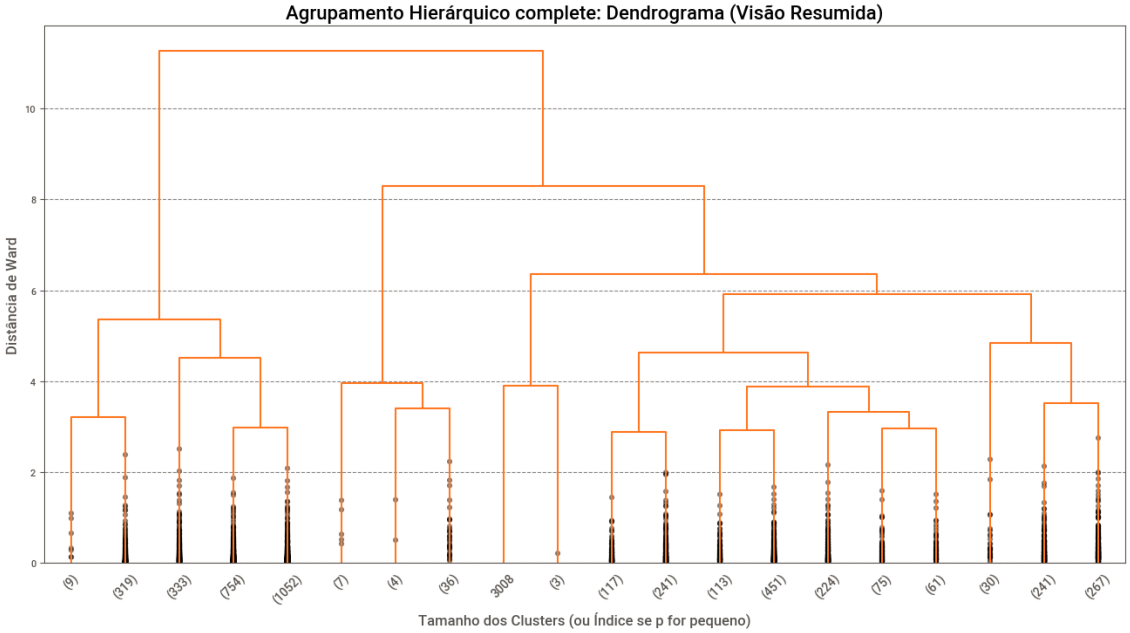
A modelagem por agrupamento hierárquico iniciou-se pela construção e avaliação de diferentes critérios de ligação (*linkage*), testando as abordagens de distância Single, Complete e Ward. Conforme adiantado na metodologia, as execuções dos métodos Single e Complete resultaram em formações indesejadas (*chaining* e partições de baixa qualidade, respectivamente) e foram descartadas, direcionando a análise definitiva para a técnica aglomerativa de Ward, que tem como premissa minimizar a variância intra-cluster a cada novo passo de fusão.

Para visualizar a estruturação progressiva das distâncias, plotou-se o Dendrograma truncado do modelo Ward com uma linha para a sugestão de divisão de clusters.

```
rfm_cols = ['Recency', 'Frequency', 'Monetary']
X_all = df_rfm_final[rfm_cols]
methods = ["ward", 'single',"complete"]

for method in methods:
    Z = linkage(X_all, method=method)

    plt.figure(figsize=(16, 8))

    # Dendrograma:
    dendrogram_plot = dendrogram(
        Z,
        truncate_mode='lastp', # condensa os dados em 'p' clusters finais
        p=20,                  # Mostra apenas os últimos 20 agrupamentos
        leaf_rotation=45.,
        leaf_font_size=10.,
        show_contracted=True,  # Mostra pontinhos pretos onde há ramos ocultos
        color_threshold=40     # Muda a cor dos ramos abaixo dessa distância
    )

    # Adicionando uma linha de corte horizontal (opcional)
    plt.axhline(y=40, color='r', linestyle='--')

    plt.title(f'Agrupamento Hierárquico {method}: Dendrograma (Visão Resumida)', fontsize=16)
    plt.xlabel('Tamanho dos Clusters (ou Índice se p for pequeno)', fontsize=12)
    plt.ylabel('Distância de Ward', fontsize=12)
    plt.grid(axis='y', linestyle='--', alpha=0.7) # Grid suave para facilitar leitura da distância

    plt.show()
```



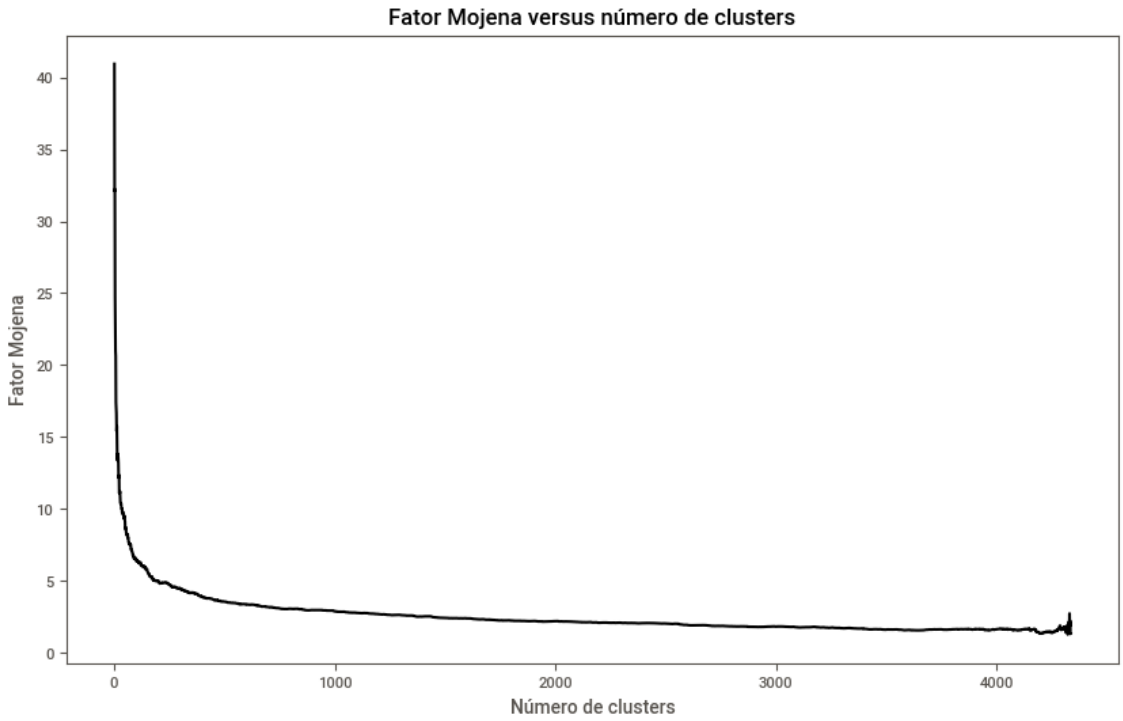




Visando determinar matematicamente o momento exato de corte (número ótimo de partições), implementou-se a Regra de Parada pelo Fator de Mojena. A avaliação matemática restrita pelo Fator de Mojena sugeriu uma divisão ótima correspondente a 77 clusters.

```
# Vamos aplicar o método o método de Mojena para o Ward
# Aplica o método de Ward para o agrupamento hierárquico
Z = linkage(X_all, method='ward')
Z
✓ 0.4s

rray([[3.46900000e+03, 3.95800000e+03, 8.26105800e-05, 2.00000000e+00],
       [7.23000000e+02, 4.24700000e+03, 3.72657990e-04, 2.00000000e+00],
       [2.63400000e+03, 3.38400000e+03, 4.18527982e-04, 2.00000000e+00],
       ...,
       [8.66700000e+03, 8.66900000e+03, 4.48943121e+01, 1.10500000e+03],
       [8.67000000e+03, 8.67100000e+03, 6.35385660e+01, 3.23300000e+03],
       [8.67200000e+03, 8.67300000e+03, 1.04827563e+02, 4.33800000e+03]],
      shape=(4337, 4))
```

```python
# Implementação do Método de Mojena
def mojena_fator(Z):
    """
    Implementa o método de Mojena para agrupamento hierárquico.
    Z: Matriz linkage do scipy
    k_factor: Valor padrão geralmente entre 1.25 e 1.50
    """
    # Distâncias de fusão (terceira coluna da matriz Z)
    distances = Z[:, 2]

    # Calcular média e desvio padrão das distâncias
    mean_dist = np.mean(distances[0:-1])
    std_dist = np.std(distances[0:-1])

    # Calcular critério de Mojena (k) para cada estágio
    # k_j = (alpha_j - mean) / std
    mojena_k = (distances[-1] - mean_dist) / std_dist

    return mojena_k

moj_factor = []
N = len(Z)

N_clusters_values = list(range(2, N))
for i in N_clusters_values:
    aux = Z[0:i, :]
    moj_factor.append(mojena_fator(aux))

# Reverse both the moj_factor list and the list of cluster numbers for plotting
moj_factor.reverse()

plt.figure(figsize=(10, 6))
plt.plot(N_clusters_values, moj_factor, color='black')
plt.title('Fator Mojena versus número de clusters')
plt.xlabel('Número de clusters')
plt.ylabel('Fator Mojena')
plt.show()
```



```python
# Função para encontrar o melhor número de clusters baseado no fator Mojena

def mojena_stopping_rule(Z, k_factor):
    """
    Implementa o método de Mojena para agrupamento hierárquico.
    Z: Matriz linkage do scipy
    k_factor: Valor padrão geralmente entre 1.25 e 1.50
    """
    # Distâncias de fusão (terceira coluna da matriz Z)
    distances = Z[:, 2]

    # Calcular média e desvio padrão das distâncias
    mean_dist = np.mean(distances)
    std_dist = np.std(distances)

    # Calcular critério de Mojena (k) para cada estágio
    # k_j = (alpha_j - mean) / std
    mojena_k = (distances - mean_dist) / std_dist

    # Encontrar o estágio onde a distância é "significativamente" maior
    # O corte é geralmente onde mojena_k > k_factor
    # Pegamos o último estágio que está abaixo do limite para cortar

    stopping_stage = np.argmax(mojena_k > k_factor) - 1

    # Número de clusters = Total de elementos - estágio de parada
    num_clusters = len(distances) - stopping_stage

    return num_clusters, mojena_k

n_clusters, k_values = mojena_stopping_rule(Z, k_factor=1.25)
print(f"Número ideal de clusters (Mojena): {n_clusters}")
```

✓ 0.0s

```
Número ideal de clusters (Mojena): 77
```

Como uma segmentação com este nível de granularidade torna-se inadequada e inviável sob a perspectiva gerencial e de negócios, optou-se por intervir na regra de parada. Apoiando-se na observação qualitativa das ramificações do dendrograma e na definição estratégica preestabelecida, configurou-se o algoritmo AgglomerativeClustering para estruturar a base em 5 agrupamentos definitivos.

```python
# Treinamento domodelo final de Hierárquico (Ward) com clusters = 5
N_CLUSTERS_HC = 5
rfm_cols = ['Recency', 'Frequency', 'Monetary']
X_all = df_rfm_final[rfm_cols]

hc = AgglomerativeClustering(
    n_clusters=N_CLUSTERS_HC,
    metric="euclidean",
    linkage="ward",
)
df_rfm_final['Cluster_Ward'] = hc.fit_predict(X_all)

# Verificando as médias do Ward para criar os nomes
perfil_hc_ward = df_rfm_final.groupby('Cluster_Ward').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': 'mean',
    'Cluster_Ward': 'count' # Apenas para ver quantos clientes há em cada um
}).rename(columns={'Cluster_Ward': 'Total_Clientes'})

print(perfil_hc_ward)
```

✓ 0.4s

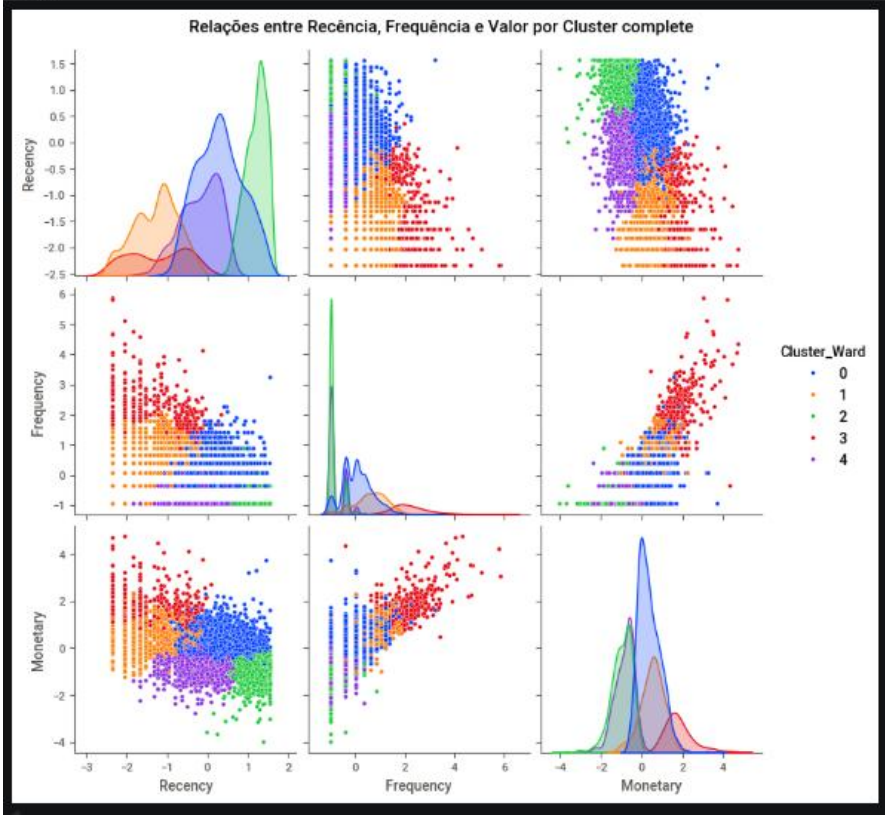
```
               Recency  Frequency  Monetary  Total_Clientes
Cluster_Ward
0             0.302681   0.091025  0.363233            1517
1            -1.272768   0.631333  0.524287             762
2             1.194048  -0.851003 -0.990959             898
3            -1.210434   2.229856  1.828660             343
4            -0.178966  -0.757703 -0.840931             818
```

Extraindo-se as médias de Recência, Frequência e Valor Monetário destes 5 agrupamentos, foi possível rotulá-los traduzindo o resultado espacial para as métricas da empresa:

- O Cluster 3 isolou o nicho que realizou compras mais recentes (recência negativa na escala) aliados a volumes muito superiores aos da média em frequência e montante financeiro, encaixando-se perfeitamente no perfil VIP.
- Os Clusters 0 e 1 mostraram recências variando de baixas a médias, mantendo um comportamento sustentável e de médio a alto volume em compras e receitas, de modo que foram fundidos para formar os clientes Fiéis.
- O Cluster 4 capturou os casos de engajamento levemente abaixo da média nas três *features*, caracterizando os clientes do segmento Promissor.
- O Cluster 2 agrupou clientes com altíssima recência (muito tempo de ausência) e a menor incidência de retornos e gastos, assinalando claramente a zona de Risco.

```python
# Gerando o gráfico de pares
sns.pairplot(df_rfm_final[['Recency', 'Frequency', 'Monetary', "Cluster_Ward"]], hue="Cluster_Ward", palette='bright', diag_kind='kde')
plt.suptitle(f'Relações entre Recência, Frequência e Valor por Cluster {method}', y=1.02)
plt.show()
```

✓ 3.0s

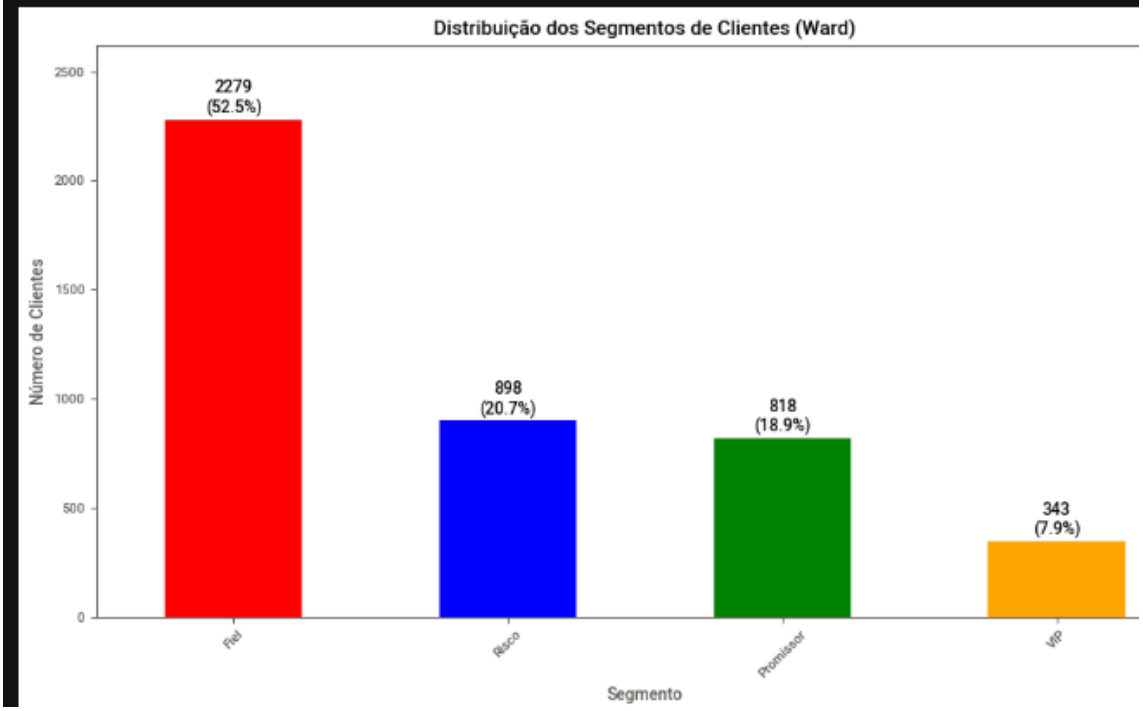



```python
# Mapeamento qualitativo
nomes_clusters_ward = {
    3: "VIP",
    1: "Fiel",
    0: "Fiel",
    4: "Promissor",
    2: "Risco"
}

# Aplicando o mapeamento
df_rfm_final['Segmento_Ward'] = df_rfm_final['Cluster_Ward'].map(nomes_clusters_ward)
```

Em sua distribuição volumétrica final, o Agrupamento Hierárquico de Ward distribuiu a base com a maioria absoluta caracterizada como Fiel englobando 52,5% (2.279 clientes), seguido pelo grupo de Risco com 20,7% (898 clientes), base Promissora com 18,9% (818 clientes) e a base VIP representando 7,9% (343 clientes). A qualificação coesa da topologia do modelo alcançou um *Silhouette Score* de 0.2331, firmando-se como o segundo melhor resultado em coesão interna neste estudo, atrás apenas do K-means.

```python
# Ver em forma de gráfica
plt.figure(figsize=(10, 6))

# Calculamos as duas métricas
contagem = df_rfm_final['Segmento_Ward'].value_counts()
porcentagem = df_rfm_final['Segmento_Ward'].value_counts(normalize=True) * 100

# Criamos o gráfico principal (usando os números absolutos no eixo Y)
contagem.plot(kind='bar', color=cores_k5)

# Adicionamos os rótulos combinados (Número + %) no topo de cada barra
for i, (label, valor_absoluto) in enumerate(contagem.items()):
    pct = porcentagem[label]
    texto = f"{valor_absoluto}\n({pct:.1f}%)" # \n quebra a linha para ficar um sobre o outro
    plt.text(i, valor_absoluto + 10, texto, ha='center', va='bottom', fontweight='bold', fontsize=10)

plt.title('Distribuição dos Segmentos de Clientes (Ward)')
plt.xlabel('Segmento')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)

# Aumentar um pouco o limite do eixo Y para o texto não cortar no topo
plt.ylim(0, contagem.max() * 1.15)

plt.tight_layout()
plt.show()
```

✓ 0.1s



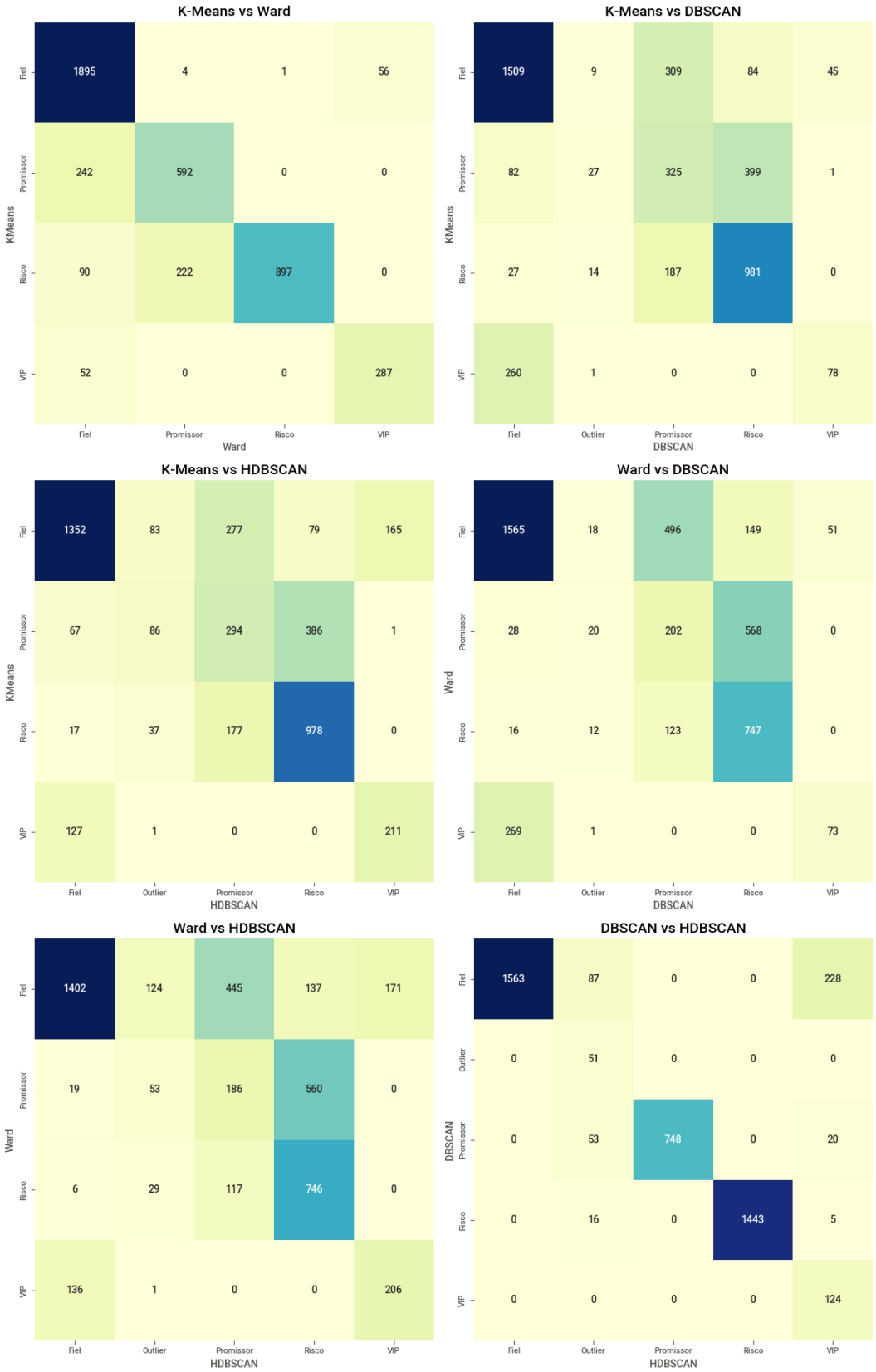
3.9:Comparação de Métodos

Para avaliar e validar os resultados finais, realizou-se a triangulação dos modelos, contrastando quantitativamente e qualitativamente as segmentações geradas pelo K-means, DBSCAN, HDBSCAN e Método de Ward.

Heatmaps de Contingência (Matriz de Consistência): Foi construída uma matriz visual com 6 gráficos de calor para comparar os agrupamentos par a par (K-Means vs Ward, K-Means vs DBSCAN, K-Means vs HDBSCAN, Ward vs DBSCAN, Ward vs HDBSCAN e DBSCAN vs HDBSCAN). Através destas matrizes de tabulação cruzada, torna-se possível observar as interseções exatas e como os clientes de um perfil qualitativo (VIP, Fiel, Promissor e Risco) migraram ou se mantiveram consistentes entre algoritmos de naturezas tão diferentes.

```
# 1. Definimos os pares de comparação
comparacoes = [
    ('Segmento_KMeans', 'Segmento_Ward', 'K-Means vs Ward'),
    ('Segmento_KMeans', 'Segmento_DBSCAN', 'K-Means vs DBSCAN'),
    ('Segmento_KMeans', 'Segmento_HDBSCAN', 'K-Means vs HDBSCAN'),
    ('Segmento_Ward', 'Segmento_DBSCAN', 'Ward vs DBSCAN'),
    ('Segmento_Ward', 'Segmento_HDBSCAN', 'Ward vs HDBSCAN'),
    ('Segmento_DBSCAN', 'Segmento_HDBSCAN', 'DBSCAN vs HDBSCAN')
]

# 2. Criamos a moldura (3 linhas, 2 colunas)
fig, axes = plt.subplots(3, 2, figsize=(12, 22))
fig.suptitle('Triangulação de Modelos: Matriz de Consistência Entre Algoritmos', fontsize=20)

# O segredo está no .flatten() aqui embaixo:
for i, (col_y, col_x, titulo) in enumerate(comparacoes):
    ax = axes.flatten()[i] # Achata a matriz 3x2 em uma lista de 6 posições

    # Criando a tabela de cruzamento
    tabela_cruzada = pd.crosstab(df_rfm_final[col_y], df_rfm_final[col_x])

    # Plotando o Heatmap
    sns.heatmap(tabela_cruzada, annot=True, fmt='d', cmap='YlGnBu', ax=ax, cbar=False)

    ax.set_title(titulo, fontsize=14, fontweight='bold')
    ax.set_xlabel(col_x.split('_')[-1])
    ax.set_ylabel(col_y.split('_')[-1])

# Opcional: Remover o último subplot se ele ficar vazio (já que você tem 5 comparações para 6 espaços)
if len(comparacoes) < 6:
    axes.flatten()[-1].axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.90])
plt.show()
```

Triangulação de Modelos: Matriz de Consistência Entre Algoritmos



Adjusted Rand Index (ARI): A matriz de cálculo do *Adjusted Rand Index* demonstrou estatisticamente a concordância global entre os algoritmos. Os resultados revelaram uma forte afinidade intrafamílias de modelos: os algoritmos baseados em centroide e variância (K-Means e Ward) apresentaram uma boa concordância mútua, atingindo um ARI de 0.648. Em paralelo, os modelos espaciais baseados em densidade (DBSCAN e HDBSCAN) mostraram uma altíssima similaridade estrutural entre si, com um ARI de 0.839. Nas intersecções entre famílias distintas (ex: K-Means vs DBSCAN), o índice oscilou em faixas mais moderadas, de 0.317 a 0.399.

```python
# --- Similaridade entre partições (Adjusted Rand Index, ARI) ---
# ARI = 1.0 partição idêntica; ~0 aleatório; negativo pior que acaso.
rotulos = {
    "K-Means": df_rfm_final["Segmento_KMeans"],
    "DBSCAN": df_rfm_final["Segmento_DBSCAN"],
    "HDBSCAN": df_rfm_final["Segmento_HDBSCAN"],
    "Ward": df_rfm_final["Segmento_Ward"]
}
nomes = list(rotulos.keys())
ari = pd.DataFrame(np.eye(len(nomes)), index=nomes, columns=nomes)
for i, a in enumerate(nomes):
    for j, b in enumerate(nomes):
        if j <= i:
            continue
        v = adjusted_rand_score(
            pd.Categorical(rotulos[a]).codes,
            pd.Categorical(rotulos[b]).codes,
        )
        ari.loc[a, b] = ari.loc[b, a] = v
print("Matriz ARI (simetrizada) entre métodos:")
display(ari.round(3))
```

✓ 0.0s

```
Matriz ARI (simetrizada) entre métodos:
```

	K-Means	DBSCAN	HDBSCAN	Ward
K-Means	1.000	0.399	0.386	0.648
DBSCAN	0.399	1.000	0.839	0.337
HDBSCAN	0.386	0.839	1.000	0.317
Ward	0.648	0.337	0.317	1.000

Comparativo de Silhouette Score: O comparativo da métrica de silhueta evidenciou que, em termos absolutos, os valores obtidos indicam uma estrutura de agrupamentos com fronteiras difusas e certa sobreposição. Valores de silhueta nesta faixa (entre 0.14 e 0.28) demonstram que os clientes não se dividem em grupos perfeitamente isolados no espaço tridimensional. Contudo, em se tratando de dados comportamentais humanos (matriz RFM), onde os padrões de consumo de fato fluem de forma contínua, uma sobreposição entre grupos é uma representação natural da realidade do negócio. Em termos relativos, o algoritmo K-means obteve a melhor coesão interna nos dados propostos, alcançando 0.2791, seguido pelo método hierárquico de Ward, com 0.2331. Para garantir justiça analítica, o índice dos métodos de densidade foi calculado desconsiderando-se as instâncias rotuladas rigorosamente como ruído. Sob esta regra, as estruturas nucleares do DBSCAN e do HDBSCAN registraram métricas coesas praticamente idênticas, de 0.1485 e 0.1483, respectivamente.

```python
sil_ward = silhouette_score(X_all, df_rfm_final['Segmento_Ward'])
sil_kmeans = silhouette_score(X_all, df_rfm_final['Segmento_KMeans'])
sil_dbscan = silhouette_score(X_all[df_rfm_final['Segmento_DBSCAN'] != 'Outlier'], df_rfm_final[df_rfm_final['Segmento_DBSCAN'] != 'Outlier']['Segmento_DBSCAN'])
sil_hdbscan = silhouette_score(X_all[df_rfm_final['Segmento_HDBSCAN'] != 'Outlier'], df_rfm_final[df_rfm_final['Segmento_HDBSCAN'] != 'Outlier']['Segmento_HDBSCAN'])

print(f"Silhueta Ward: {sil_ward:.4f}")
print(f"Silhueta K-Means: {sil_kmeans:.4f}")
print(f"Silhueta DBSCAN (sem outliers): {sil_dbscan:.4f}")
print(f"Silhueta HDBSCAN (sem outliers): {sil_hdbscan:.4f}")
```

✓ 1.2s

```
Silhueta Ward: 0.2331
Silhueta K-Means: 0.2791
Silhueta DBSCAN (sem outliers): 0.1485
Silhueta HDBSCAN (sem outliers): 0.1483
```

Distribuição Percentual e Volumétrica: A consolidação da volumetria atestou a coerência dos modelos frente às expectativas de negócio preestabelecidas. O perfil "Fiel" estabeleceu-se de forma majoritária em todos os testes, aglomerando entre 1.563 clientes (no HDBSCAN) e 2.279 clientes (no Ward). O segmento "Promissor" revelou-se extremamente estável perante todos os vieses matemáticos, variando uma faixa muito estreita entre 748 e 834 clientes em todo o estudo. A camada "VIP" também exibiu coerência considerável: o HDBSCAN encontrou 377 clientes neste perfil estratégico, seguido de perto pelo Ward (343) e o K-Means (339), tendo o DBSCAN assumido uma postura de qualificação mais estrita nesta borda, com apenas 124 clientes. Diferenciando-se das técnicas de clusterização tradicionais, o DBSCAN e o HDBSCAN comprovaram seu valor na detecção de anomalias rígidas, isolando, respectivamente, 51 e 207 clientes irrecuperáveis sob a rubrica rigorosa de "Outliers".

```
# Distribuição de clientes por método (contagens por segmento)
resumo_dist1 = pd.DataFrame({
    'K-Means': df_rfm_final['Segmento_KMeans'].value_counts(),
    'DBSCAN': df_rfm_final['Segmento_DBSCAN'].value_counts(),
    'HDBSCAN': df_rfm_final['Segmento_HDBSCAN'].value_counts(),
    'Ward': df_rfm_final['Segmento_Ward'].value_counts()
}).fillna(0).astype(int)

print("Distribuição de clientes por método:")
display(resumo_dist1)
```

✓ 0.0s

```
Distribuição de clientes por método:
```

	K-Means	DBSCAN	HDBSCAN	Ward
Fiel	1956	1878	1563	2279
Outlier	0	51	207	0
Promissor	834	821	748	818
Risco	1209	1464	1443	898
VIP	339	124	377	343

4: Conclusão

O presente estudo cumpriu com êxito o objetivo de aplicar técnicas de aprendizado de máquina não supervisionado para segmentar clientes de um e-commerce varejista. A transformação dos dados transacionais brutos em uma matriz comportamental RFM (Recência, Frequência e Monetário) provou-se uma estratégia eficaz, permitindo que algoritmos matemáticos traduzissem dados soltos em personas operacionais de alto valor estratégico: VIP, Fiel, Promissor e Risco.

Ao avaliar as abordagens metodológicas quantitativas, observou-se que os valores absolutos do *Silhouette Score* apontam para uma estrutura de clusters com sobreposição moderada (com o pico do estudo atingindo 0.2791 pelo K-Means). Essa métrica, embora não demonstre agrupamentos perfeitamente apartados no plano

matemático, é condizente com a realidade de dados transacionais e comportamentais de clientes, cujos hábitos de compra se misturam em um espectro contínuo em vez de formarem categorias engessadas. Ainda assim, no comparativo entre as técnicas, conclui-se que algoritmos baseados em centroides e variância (K-Means e Método Hierárquico de Ward) apresentaram o melhor desempenho relativo em termos de coesão interna para esta base de dados. O K-Means obteve o *Silhouette Score* mais alto do estudo (0.2791), seguido pelo método de Ward (0.2331). Além disso, a métrica de *Adjusted Rand Index* (ARI) validou estatisticamente uma forte concordância estrutural entre estes dois modelos (ARI de 0.648)

Por outro lado, os modelos espaciais baseados em densidade (DBSCAN e HDBSCAN) confirmaram o seu grande diferencial analítico: a capacidade intrínseca de isolar ruídos no espaço amostral. Embora tenham apresentado índices de validação interna de silhueta ligeiramente mais baixos para seus núcleos (ambos na faixa de 0.148), eles foram capazes de identificar e segregar anomalias estritas (identificando 51 e 207 clientes *outliers*, respectivamente). Estes modelos também demonstraram uma altíssima concordância entre si, atingindo um ARI excelente de 0.839.

Do ponto de vista de negócios, a triangulação dos modelos revelou uma consistência notável na identificação do comportamento dos clientes. O segmento "Fiel" consolidou-se como a maior fatia da base de clientes em todos os testes (representando entre 1.563 e 2.279 clientes), e o grupo "Promissor" revelou-se extremamente estável frente a qualquer viés matemático (variando apenas entre 748 e 834 clientes).

Diante dos resultados alcançados, o modelo K-Means com K=5 (ajustado para 4 perfis de negócio) destaca-se como a recomendação final mais equilibrada para implantação pela empresa. Ele entrega a maior qualidade de particionamento matemático e fornece agrupamentos intuitivos que viabilizam o direcionamento imediato de campanhas de marketing, programas de fidelidade para os "VIPs" e "Fiéis", estímulos de compra para os "Promissores" e ações de resgate para clientes em "Risco".